

Electronics and Computer Science
Faculty of Engineering and Physical Sciences
University of Southampton

Ash-Hab Abbasi
28.04.2025

2D Animation Package w/ Inverse
Kinematics

Project supervisor: David Millard
Second examiner: Adam Prugel-Bennett

A project report submitted for the award of
MEng Computer Science

Abstract

The purpose of this project is to provide a simple and intuitive 2D animation software with inverse kinematics (IK) for natural character control. It will let users at any skill level to animate stick figures and export them for sharing on social media.

I've created a minimum viable implementation of the product, as well as produced various animations with the software. It incorporates IK for easy natural character control, as well as separate timelines for each stick figure. Interpolation is a key feature which can be used to fill in the frames between the created keyframes, preventing users from having to animate every frame manually. I've finished the majority of the requirements. Originally, there were ideas to implement a physics system for more enhanced motion, however, it wasn't possible to add this along with some other requirements in the time given for this project.

Considering the future potential of this project, the software can be finished off with the rest of the requirements, and any new features (e.g. undo button, or the ability to mirror the characters) discussed during the evaluation phase can be added to it. The animation package can then be distributed for anyone to use, allowing new users to enter the animation field with a tool which makes it much easier to animate.

Statement of Originality

- I have read and understood the ECS Academic Integrity information and the University's Academic Integrity Guidance for Students.
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.
- I have acknowledged all sources, and identified any content taken from elsewhere.
- I have not used any resources produced by anyone else.
- I did all the work myself, or with my allocated group, and have not helped anyone else.
- The material in the report is genuine, and I have included all my data/code/designs.
- I have not submitted any part of this work for another assessment.
- My work **did** involve human participants, who all consented by reading the Participant Information Sheet and filling a digital consent form (Ethics reference number: ERGO78677.A2).

Contents

Abstract	2
Statement of Originality	3
Contents	4
1 Introduction	6
1.1 Problem	6
1.2 Goals	6
1.3 Scope	6
2 Background research of the project	7
2.1 Literature Review	7
2.1.1 Introduction	7
2.1.2 Skeletal Animation and Kinematics	7
2.1.3 Inverse Kinematics Methods for Animation	8
2.1.4 Timelines and Layering	8
2.1.5 Interpolation and In-Between Frames	9
2.1.6 Physics in Animation	9
2.1.7 Challenges and Innovations	10
2.1.8 Conclusion	11
2.2 Systematic Review of Software	12
2.2.1 Existing Tools	12
2.2.2 Linking Back to the Project	13
3 Design of the project	13
3.1 Functional and Non-Functional Requirements	13
3.2 Wireframes	15
3.3 UML Activity Diagram	16
4 Implementation	16
4.1 Justifying technology choices	17
4.1.1 Java	17
4.1.2 Python	17
4.1.3 Final Choice	18
4.2 Technology Stack	18
4.3 Software Architecture (Model-View-Controller)	19
4.4 Core Component Implementation	22
4.5 Inverse Kinematics (FABRIK implementation)	23
4.6 Visual Presentation	24
4.7 Implementation Summary	27
5 Testing	28
5.1 Testing Methodology	28
5.2 White-Box Testing Procedures	28
5.3 Black-Box Testing (Requirements Verification)	29
5.4 Discussion of Testing Results	30
5.5 Testing Summary	31

6 Evaluation	32
6.1 Inverse Kinematics (IK) as a Core Strength	32
6.2 Timeline Management and Workflow	33
6.3 User Interface (UI) and Intuitiveness	34
6.4 Missing Features and Future Improvements	34
6.5 Evaluation Summary	35
7 Project Management	35
7.1 Development Approach and Methodology	36
7.2 Project Management Tools	36
7.3 Risk Analysis and Mitigation	37
7.4 Project Planning and Scheduling (Gantt Charts)	37
7.5 Personal Reflection	39
8 Conclusion	39
8.1 Summary of Achievements	39
8.2 Limitations	40
8.3 Future Work	41
References	42
Appendices	43
Appendix 1: Systematic Review Descriptions	43
1.1 Pivot Animator	43
1.2 OpenToonz	44
1.3 Synfig	45
1.4 Moho	46
Appendix 2: Functional and Non-Functional Requirements	47
2.1 Functional Requirements:	47
2.2 Non-Functional Requirements:	49
Appendix 3: Wireframes	50
Appendix 4: Requirements Assessment Table	52
Appendix 5: Risk Analysis	57
Appendix 6: Initial Project Brief (Progress Report)	58
Problem	58
Goals	58
Scope	58

1 Introduction

1.1 Problem

The popularity of animation as a storytelling tool is only increasing [1]. Additionally, the demand for animation tools has started to rise. Affordable, user-friendly versions of these resources are not readily accessible to independent artists, students, or hobbyists. Inverse kinematics and basic physics systems, which would allow users to create smooth, natural motions with more intuitive animation control, are absent from current home animation programs. Animators without access to specialized software would find it challenging to produce dynamic, lifelike motions without these features. The challenges that currently come with creating animations may also discourage aspiring artists from pursuing careers as storytellers.

1.2 Goals

The project's objective is to produce a 2D animation package with inverse kinematics in an intuitive interface that will enable users of all skill levels to create fluid, natural motions with stick figures constructed using simple shapes. This increases 2D animation's accessibility to a wider range of users. Users can export their projects from the software to MP4 and GIF formats, and thus also share their work more readily as a result.

1.3 Scope

With inverse kinematics (IK) as an option for simpler character control, this project concentrates on creating a 2D animation toolkit that will enable users to build simple stick figure animations using puppet-style controls [2]. More complicated physics components, like momentum or complex force simulations, are outside the scope of this project, but as a stretch goal, basics like collision management or gravity will be investigated later in this report. Although commercialization might be an option, the main goal is to develop a working prototype that can be shared for feedback and give aspiring animators a tool that will help them connect the gap between accessibility and more complex tools like inverse kinematics and physics.

2 Background research of the project

2.1 Literature Review

2.1.1 Introduction

People have been using various media to convey stories for decades, from the earliest cave paintings to modern motion pictures. Unlike when individuals used written words and pictures to explore stories, the emergence of animation has allowed humans to express their creativity through motion, allowing narratives to be portrayed with deeper emotional depth [1]. Animation's development has transformed a variety of areas, from education to entertainment.

As noted in Tom Sito's "The History of Computer Animation," the area has seen substantial transformation, especially with the development of computer graphics. The multibillion-dollar industry began when Ivan Sutherland developed "Sketchpad," the first genuine animation application, on a Cold War-era computer [3].

This literature review looks at how animation has changed in the digital era, focusing on how techniques like inverse kinematics, skeletal animation and physics have enhanced animators' workflows and provided efficient ways to create realistic character motion. I will highlight the limitations of existing tools, particularly about making these techniques accessible to beginners and provide the context for this project, a 2D animation package designed to help aspiring artists enter the field of animation.

2.1.2 Skeletal Animation and Kinematics

Because early animators had to draw each frame by hand, it required a lot of time and effort to make even a short sequence. As digital technologies advanced, animators were able to produce character motion using new and improved techniques [4]. Skeletal animation revolutionized the film and television industries. This method creates a virtual "skeleton" of a character and then the animator can pose the character to create motion [5], allowing them to concentrate on the character's movement to tell a story instead of worrying about how each animation frame should appear. As a result, the animation process runs more smoothly and consistently.

Kinematics, the study of motion without taking into account the forces involved [6], is what gives skeletal animation its effectiveness. Animators have two ways to control the joints of the character's skeleton: forward kinematics (FK) and inverse kinematics (IK) [2]. Using forward kinematics, animators manipulate each character joint to move it to the appropriate

location. Because of its ease of use, this technique is frequently utilized as the norm in the majority of animation packages. Inverse kinematics goes one step further by enabling animators to use an "end effector" to reach the desired position. The end of a joint chain, such as a hand or foot, is usually the end effector. As the animators change the location of this end joint, the inverse kinematics algorithm automatically updates the prior joints to match. In the case of a hand, the system would adjust the elbow and shoulder locations to ensure that the arm was appropriately aligned [7]. This makes animating intricate movements, such as character walking, easier. The animator no longer needs to position the leg, knee, and foot separately; now, they just need to move the foot, and the other leg joints will take care of themselves.

2.1.3 Inverse Kinematics Methods for Animation

Animation's use of inverse kinematics (IK) has advanced significantly, with many implementations presenting unique benefits and drawbacks. Conventional IK solvers, like Jacobian-based approaches, use equation systems to calculate the joint angles necessary to reach a given end effector position. Because of their mathematical precision and ability to calculate the joints' angles and positions accurately, these solvers are highly regarded. Nevertheless, the computational cost of using them effectively is high. Notably, Jacobian-based techniques are less appropriate for real-time performance [10] because of their high computing cost [6].

The Forward and Backward Reaching Inverse Kinematics (FABRIK) approach, on the other hand, offers a more computationally effective alternative. FABRIK iteratively modifies joint locations by reaching forwards and then backwards along the chain until the desired end effector position is reached, as opposed to using matrix calculations and differential calculus [11]. Applications like interactive video games and real-time animation find this iterative technique appealing since it lowers processing time and enables quick, real-time changes. Although the simplicity of FABRIK is a strength, it may occasionally sacrifice precision.

Selecting one of these IK approaches should take into account both the application context and computational efficiency. While FABRIK's speed and ease of integration may be more suitable for real-time systems that may accept a slight accuracy tradeoff, Jacobian-based techniques' rigorous, although resource-intensive, nature may be justified in situations requiring great precision.

2.1.4 Timelines and Layering

In digital animation processes, controlling several animated parts is essential. In order to simplify the compositing process and enable 'non-destructive' editing, modern software permits every graphic to have its own timeline, allowing layers or independent objects to be modified without interfering with one another [8]. For example, animators can manage the pacing and aspects of each character separately by giving each animated figure its own

timeline. This facilitates complex interactions where overlapping or sequential movements may be easily managed.

2.1.5 Interpolation and In-Between Frames

Nowadays, we may create in-between frames automatically by interpolating between two "keyframes," rather than manually editing each frame, thanks to modern methods [2]. By guaranteeing frame-to-frame consistency with mathematical computation, this method speeds up motion production. Since the poses in skeleton animation are based on joint rotation, this works well. A seamless transition between the poses can be achieved by the user setting the joint rotations for the two start and end keyframes, then the system calculates the in-between frames.

However, merely employing linear interpolation to move between keyframes might produce motions that seem rigid and robotic. As a result, animation may feel off-putting due to the artificial, mechanical movement. To solve this issue, animators frequently incorporate "easing functions," also known as timing functions or motion curves, to avoid the robotic feeling by introducing gradual acceleration and deceleration. This method produces more natural transitions that more closely resemble motion dynamics in the real world. Ease-in and ease-out [12] are common easing types that shape the interpolation curve to give characters more realistic movement. For instance, ease-in enables a motion to begin slowly and gradually accelerate, whereas ease-out accomplishes the opposite. For a more seamless transition, ease-in-out combines the two effects. In addition to improving visual attractiveness, these methods provide animators more control over timing and emphasis, which helps them convey subtleties like anticipation and follow-through. According to traditional animation principles, which are covered in detail in "The Illusion of Life" by Frank Thomas and Ollie Johnston, the nuances of motion, like timing and spacing, are essential to producing believable animated performances.

2.1.6 Physics in Animation

To further improve the quality of animated motions, physics-based techniques have emerged, building on the developments in keyframe interpolation and skeleton animation. Simple physics simulations may result in more realistic interactions while requiring less manual labour.

Instead of calculating forces and integrating differential equations, Position Based Dynamics (PBD) enforces limits on the positions of objects, making it an efficient technique for real-time simulation. Even in complicated systems, this method ensures robustness and computational efficiency by iteratively adjusting their positions to fulfil requirements (similar to how FABRIK operates) [13]. Because of its established reliability, PBD is a powerful

system to include in simplified animation toolkits, where usability and real-time feedback are crucial.

The term "secondary motion" describes the organic, delayed movement of parts that come after a primary action, such as a limb's sway (for example, a dog's tail as it runs). In animation, this subtlety is essential for expressing realism and emotional depth. Computational methods for effectively simulating secondary motion have been developed recently [14], producing follow-through effects automatically that improve overall animation quality without requiring the animator to perform extra keyframing or labour.

Using physics-informed neural networks (PINN) to interpolate between keyframes is a new, complicated method in the field. Neural networks are trained in this framework to understand physical limitations and predict natural in-between frames that encapsulate motion dynamics, including momentum, acceleration, and deceleration [15]. This method has the potential to automate some aspects of the animation process while maintaining the motion's physical authenticity and provides a sophisticated way to create fluid, realistic transitions.

2.1.7 Challenges and Innovations

Inverse kinematics (IK) has revolutionized character animation, however, most animation software still makes it difficult and technical to use. Animators frequently need to understand rigging, the concept of constraints, and the hierarchical relationships between the joints to create even a simple skeleton with IK features [2]. Beginners may be deterred from using what could otherwise be a very useful tool by its intricacy, especially if they want to concentrate on minimalist styles like stick figure animations. My solution aims to quickly address this problem by making IK the default for stick figure animations, removing the need for the user to have rigging knowledge or construct hierarchical joint structures.

Beginners and hobbyists frequently cannot access advanced IK systems since they are typically restricted to costly professional software. An example of software that has great IK features but is unaffordable is Moho. For stick figure animation, there are free tools like Pivot, however, they rely on Forward Kinematics (FK) and lack IK features (for a comprehensive analysis of the current software, see **Section 2.2**). Animators may find it more difficult and time-consuming to work with FK. My project aims to bridge this gap by providing a free alternative that adds IK as a crucial element. We can focus on the animation process with stick figures without sacrificing IK's power.

Prebuilt skeletons and streamlined IK techniques eliminate the need for manual rigging, making IK accessible to novice users. By making IK accessible to novices and empowering them to create fluid movements, my project - which focuses on stick figures with IK as the

standard - promotes the field of animation by giving animators an alternative to FK-only tools like Pivot without the hassle and cost of professional-grade software. With no technical restrictions, I can allow animators to experiment with a new animation style that they may not be accustomed to.

Although physics-informed neural networks (PINN) and Position Based Dynamics can be very useful in an animation environment, in a 2D Animation toolkit these could be overly complicated to include as there are too many stylistic and creative choices that animators can make with 2D animation which don't follow the natural laws of physics. For example, these systems may not work properly if a character is supposed to be running on the spot, as they're not going anywhere. To solve this, making physics 'optional' is very important. This way, the animator can choose to employ realistic physics for certain scenarios such as simulating gravity for a ball rolling, while still being able to keep the possibility for stylized sequences such as a character running in place.

2.1.8 Conclusion

Storytelling has been revolutionized by animation, which enables concepts to be communicated through motion instead of words or static images [16]. With the shift from traditional hand-drawn methods to advanced digital workflows, two new techniques have emerged: skeletal animations and inverse kinematics. These made the process of animating more efficient and allowed animators to create realistic motion more rapidly. But even with these technological developments, there is still a lack of tools for inexperienced animators, as many of the animation programs that are currently available with these features are too expensive and complicated. (see **Section 2.2**).

Major software programs like Moho offer the use of inverse kinematics, numerous layers, and configurable timelines, but these features are still restricted to expensive solutions. Free tools like Pivot, on the other hand, only provide basic forward kinematics, limit interpolation choices, and only provide a single timeline for every animatable element, making it more challenging to time the motions of various parts together.

This project offers an innovative approach by creating a brand-new 2D animation application that incorporates inverse kinematics by default for manipulating stick figures. Each stick figure has its own timeline so animators can independently modify their timings. They can also use several interpolation techniques (ease-in, ease-out, and ease-in-out) to produce more organic, dynamic movements. These improvements address the accessibility and flexibility issues with current entry-level tools by supporting non-destructive editing and streamlining complex compositing. By emphasizing user-friendly interfaces with a free prototype, this tool lowers the entrance barrier for novice animators and bridges the gap between free tools and expensive software, enabling users to high-quality animations with minimal technical knowledge.

2.2 Systematic Review of Software

2.2.1 Existing Tools

I examined and contrasted existing software that uses puppet-style animation techniques for this project.

I've selected Pivot Animator, OpenToonz, Synfig Studio, and Moho since their processes all could include puppet-based techniques. Moho is a commercial tool, although Pivot, Synfig, and Opentoonz are all free. Pivot is an easy-to-use tool that anyone can use, however, it can only be used with forward kinematics, and the characters you can make are limited because it is based on stick figures. While OpenToonz and Synfig emphasize more conventional methods of animation with drawing tools, they also offer possibilities for inverse kinematic puppet-based rigging. However, because the workflow is centred on conventional animation approaches, the puppet-based tools may appear basic. Professionals use Moho, a sophisticated yet pricey piece of software. In addition to physics components, it features sophisticated rigging and inverse kinematics algorithms. However, this has the drawback of being less accessible than the other tools due to its high cost and steep learning curve. The broad overview of the four programs is displayed in **Table 1**. Refer to **Appendix 1** for a more thorough explanation of each piece of software.

Table 1:

Software (with puppet based animation)	Compatible operating systems	Inverse Kinematics	Physics	Complexity for Beginners	Export Formats	Character Design Options	Cost
Pivot Animator	Windows only	no	no	Very easy (Simple interface, minimal tools, beginner-fri endly)	GIF, PNG	Very Limited (Lines and shapes only)	Free
OpenToonz	Windows, Mac	yes	no	Moderate (steeper learning curves)	MP4, GIF, PNG	High (Freeform Drawing and Rigging)	Free (open source)

Synfig Studio	Windows, Linux, MacOS	yes	no	Moderate (steeper learning curves)	AVI, GIF, PNG	High (Freeform Drawing and Rigging)	Free (Open source)
Moho	Windows, MacOS	yes	yes	Difficult (Advanced toolset; time investment required)	SWF, AVI, PNG	High (Freeform Drawing or Rigging)	Paid

Complexity: determined by the software's tool count, learning curve, and need for particular skills (e.g., how to enable the physics simulations or use the rigging features).

Character design options: The freedom to animate characters or additional elements, whether that freedom is restricted to lines or forms, allows for freeform drawings, or depends on whether the software has rigging capabilities.

2.2.2 Linking Back to the Project

Numerous shortcomings in these current tools have been noted by this systematic review. Although Pivot Animator lacks sophisticated capabilities like inverse kinematics and physics tools, it is quite user-friendly for novice animators of all skill levels. This restricts the software's potential. These sophisticated features are available in the other packages, but they come with steep learning curves and at the expense of accessibility. Because Moho is designed for experts, it has the steepest learning curve even with all the sophisticated capabilities. By combining Pivot's ease of use and accessibility with more sophisticated capabilities like inverse kinematics, my project hopes to close the gap between accessibility and sophisticated tools and enable animators to express their creativity with fewer limitations.

3 Design of the project

3.1 Functional and Non-Functional Requirements

I created a table with the list of functional and non-functional requirements, sorted according to MOSCOW prioritization, in order to keep track of the requirements for this project. I have divided the 14 functional requirements into the following sections: physics system, interface, inverse kinematics, 2D animation creation, exporting animations, and customization. In addition, I have separated the 10 non-functional needs into four categories: accessibility, usability, reliability, and performance.

One of the functional requirement sections, 2D Animation Creation, is displayed in **Table 2**. **Appendix 2** contains the complete list of requirements for each section.

Table 2:

	Must have	Should have	Could have	Won't have
2. 2D Animation Creation	a. Enable users to edit pre-built stick figures by moving the joints	e. Add features such as path-based interpolation editor for smoother animation	g. 3D Camera for easy view of different perspectives of the stick figures	
	b. Ability to animate the stick figures using keyframes			
	c. Support animating multiple stick figures in the same project	f. Add onion skinning to preview the previous and next keyframes (relative to currency keyframe).		
	d. Enable real-time playback of animations within the software			

3.2 Wireframes

Wireframes are low-fidelity graphic depictions of potential user interfaces that are used as a development blueprint. Without concentrating on specifics like the graphics, they aid in conveying the arrangement and capabilities of the screens. I can make sure that the interface complies with the project's usability objectives by utilizing wireframes to sketch out the software's visual structure. I have created two wireframes for this project: one for the character editor and one for the main interaction interface. **Figure 1** is a wireframe for the main interaction interface, presenting the software's layout.

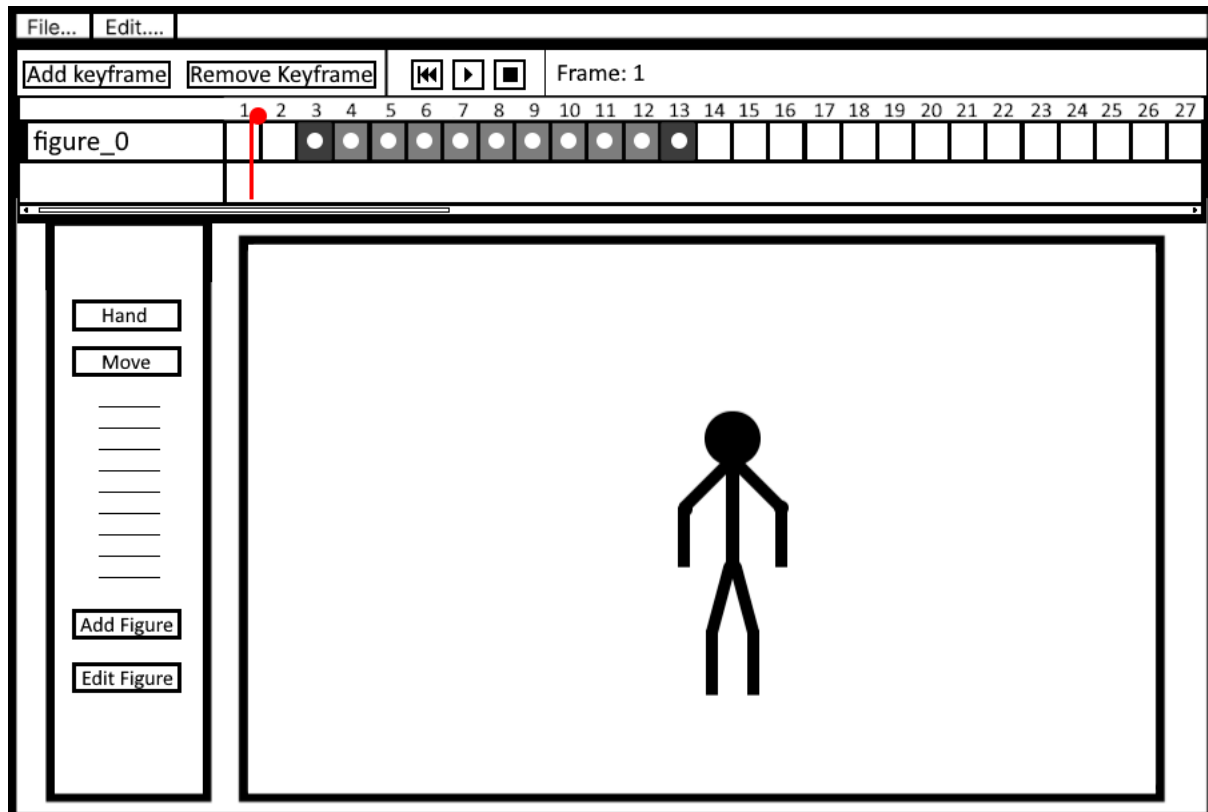


Figure 1: Wireframe of the main interface

The timeline is at the top, the toolbar is on the left, and the primary canvas is in the centre. There are several buttons on the toolbar for adding and modifying stick figures, such as an “add Figure” button, or buttons to choose whether to move the stick figure or pose it. The buttons to “add keyframe,” “remove keyframe,” “rewind,” “play,” and “stop” are conveniently located in the upper left corner. Only when there is a keyframe to delete will the “Remove keyframe” button be clickable, and only when the animation is playing will the “stop” button be clickable. The timeline at the top resembles the OpenToonz timeline, which has tiny rectangles for each frame. To draw attention to them, the keyframes and interpolated frames are coloured differently. To navigate the timeline, a draggable timeline cursor is available. Additionally, there will be a scroll bar beneath the timeline for left and right navigation. The “File” button in the top menu bar will allow you to save the project, open existing projects, or start a new one. The “Edit” button will let you change the project settings.

Appendix 3 has the other wireframe, which is the character editor.

3.3 UML Activity Diagram

One kind of Unified Modelling Language (UML) diagram that is used to illustrate the possible user workflow in the system is an activity diagram. Activity diagrams help you understand how different components function together.

A UML activity diagram for the system is displayed in **Figure 2**. From the beginning of an animation project until the creation and generation of the keyframes, this illustrates how the system behaves in terms of the production pipeline. This UML Diagram was created at the start of the project, which means there are some features that are missing (e.g. “character editor window”) from the final implementation (See **Section 7: Project Management**).

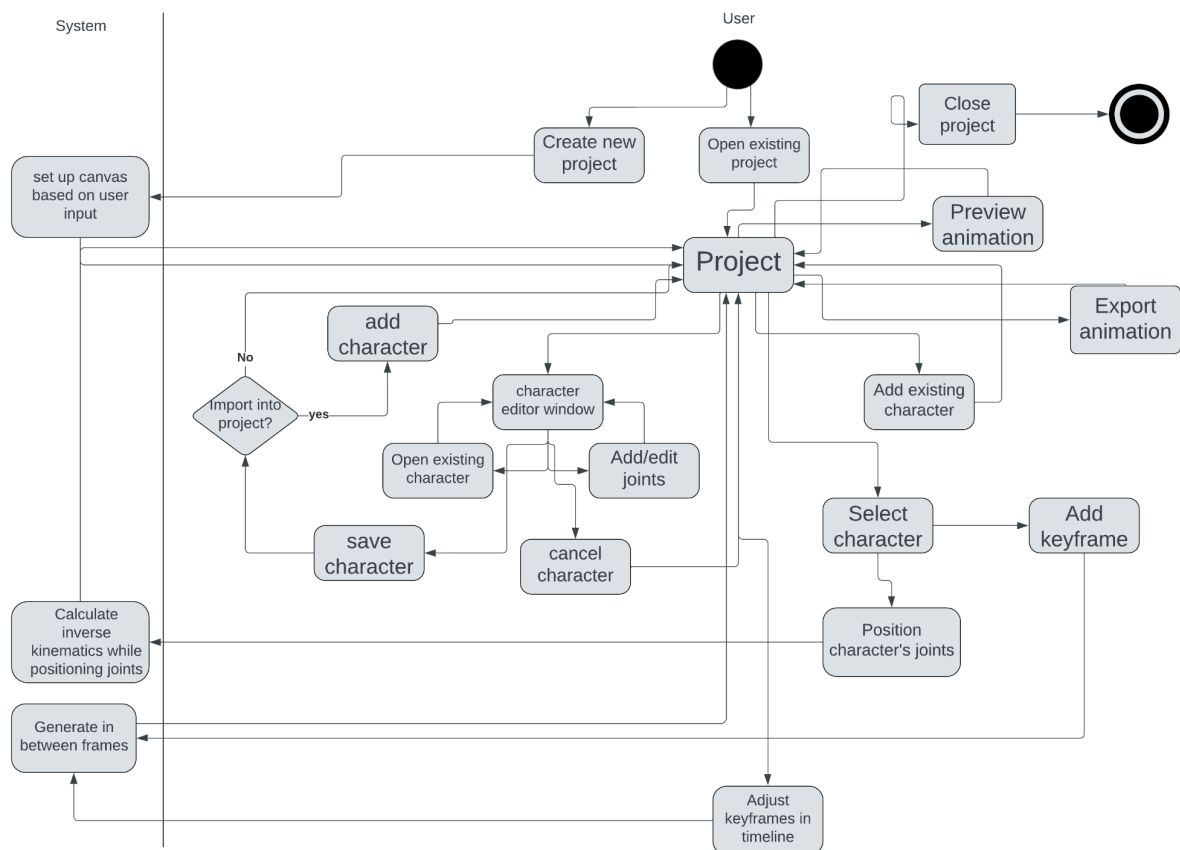


Figure 2: Activity Diagram

4 Implementation

This section describes how the 2D Animation Package was built, including the technologies used, the software architecture, how important parts operate internally, and how fundamental algorithms like Inverse Kinematics were implemented.

The main product of this project is a working 2D animation program that lets users use keyframing techniques to make and work with stick figures. Multiple stick figures can be added in, keyframes can be added, removed, or moved on a timeline, motion can be interpolated between keyframes using a variety of easing functions, and figures can be posed naturally using Inverse Kinematics (IK) by dragging end-effectors (hands and feet). It also has features for loading and saving animations as well as exporting them to popular file types like PNG, GIF, and MP4.

4.1 Justifying technology choices

In this section, I'll go over two different programming languages that I could have used to create the software during the development stage. After comparing their advantages and disadvantages, I'll explain why I ultimately chose Java.

4.1.1 Java

Java is a popular desktop application programming language with extensive documentation. It has a strong support for object-oriented programming, which is helpful when organizing elements like the stick figures' skeletal system. Numerous libraries for mathematical operations with useful documentation are available in Java. This can be helpful when calculating the inverse kinematics (IK) system. If physics simulation was a part of the project, or worked on in the future, Phys2D, a 2D physics library for Java, could be helpful.

The disadvantage of Java is that it can be more difficult to create visually appealing and modern graphical user interfaces (GUIs) than some other languages; this is important for usability and accessibility reasons. JavaFX helps sort this issue out. It is a modern toolkit for Java graphical user interface programming, which is already familiar to me. In earlier programming projects for the course, I have made use of both Java and JavaFX, which means this choice has the benefit of familiarity.

4.1.2 Python

In a similar vein, Python has a vibrant community and extensive documentation, but the main advantage of Python is that its syntax is simpler to read and write than Java's. This might shorten the time needed for development. Instead of worrying about minor details, I could concentrate on putting functionality into place.

A Python package called Pygame can be used to create drawing and animation tools. Other useful frameworks for creating user-friendly GUIs are PyQt and Tkinter. IK equations can be solved with the aid of the mathematical libraries Numpy and SymPy. If the project includes physics in the future, two different 2D physics engines can be utilized: Pymunk and PyBox2D.

Python is slower than Java for computation-intensive tasks like storing all the frames a user could create while animating or for real-time IK or physics simulations. Python's slower runtime may cause performance problems when handling large animations because it is not as low-level as Java. The program will need a strong storage and retrieval system for the animation to run successfully. Java's low-level features may make this easier than Python. Python relies heavily on libraries since it lacks a powerful native graphics framework like JavaFX.

4.1.3 Final Choice

Python is easy to program with, but Java's performance, familiarity, and strong ecosystem are why I went with it. Because Java compiles the program, it executes more quickly, which is crucial for real-time features like physics simulations and IK, as well as for packaging the application up and distributing it for animators to use. With Java, I have easier access to the computer's memory because it is a lower-level language than Python, which enables me to better manage the system for storing and retrieving user-created keyframes.

My past knowledge of Java and JavaFX helps reduce development risks and accelerate the process. Since this will be a desktop project, Java already offers a GUI framework designed for desktop applications, which is helpful. The project's data management and mathematics requirements are likewise met by Java's libraries.

In light of the project's objectives of developing a 2D animation package that is user-friendly, dependable, and performant, I have used JavaFX's GUI and animation features in conjunction with Java's effective runtime to produce a tool that satisfies the demands of aspiring animators while preserving scalability for possible future developments.

4.2 Technology Stack

The following technologies were utilized primarily in the development of the application:

Programming Language: Java (JDK 8) - selected due to its vast standard library, strong object-oriented features, and language familiarity. Because of its structure, it works well with design patterns like Model-View-Controller (MVC).

UI Framework: JavaFX - chosen as the toolkit for the graphical user interface. With its extensive collection of UI components (Buttons, Menus, ScrollPanels), layout managers (BorderPane, VBox, HBox), and a robust Canvas API appropriate for 2D graphics, JavaFX offers an innovative Java user interface platform. Instead of using FXML, the user interface was built programmatically, giving precise control over the wiring and component construction.

Build & Dependency Management: Maven - used to package the application, manage project dependencies, compile code, and provide a repeatable build procedure.

JSON Processing: Jackson Databind - used for the save/load functionality (SaveManager, AnimationSaveData) by serializing and deserializing the animation state (keyframes, figure data, and animation settings) to JSON format. It is appropriate for this task due to its performance and ease of usage.

Video Encoding: JavaCV (FFmpeg Wrapper) - used to make exporting animations to the MP4 video format easier in the MP4Encoder class. By offering access to the underlying FFmpeg library, JavaCV makes it possible to create video streams and encode frames.

Logging: Log4j 2 - included to provide reliable and adjustable logging across the course of the application lifecycle, which facilitates debugging and behavior monitoring.

The selection of Java and JavaFX offers a strong basis for a desktop application, while libraries such as JavaCV and Jackson effectively manage particularly demanding tasks like video export and save/load functions.

4.3 Software Architecture (Model-View-Controller)

The Model-View-Controller (MVC) architectural pattern was used in the application's design to provide modularity, maintainability, and testability. The application is divided into three interrelated components by MVC, which encourages a separation of concerns:

- **Model:** contains the state, business logic, and essential data of the program (e.g., figure structures, animation data, IK algorithms). It handles the data directly and is not reliant on the user interface.
- **View:** In charge of rendering the user interface and presenting the data from the model to the user (e.g., showing the canvas with the figures, as well as the timeline). When the data changes, it updates itself by observing the Model (typically through the Controller). It should have very little logic.
- **Controller:** serves as a bridge connecting the View and the Model. It receives user input from the View (such as button presses and mouse clicks), converts it into actions, modifies the Model appropriately, and chooses the relevant View response.

By keeping UI logic apart from data management and essential computations, this division improves code organization, helps with separate component creation and testing, and overall increases maintainability.

The MVC components in this 2D Stick Figure Animator application are organized as follows:

Model: Describes the data and main logic of the application. The user interface has no bearing on this layer. Important elements consist of:

- **TimelineModel:** Controls the animation state for every figure, including keyframes, interpolation segments, current frame, and frames per second. includes the fundamental interpolation logic.
- **FigureManager:** Provides access methods and maintains the selection state for the collection of StickFigureModel objects.
- **StickFigureModel:** Shows the structure (points, segments, IK chains, constraints) and behaviour (IK solution, constraint application) of a single stick figure.
- **Figure Definitions (FigureDefinition, PointDefinition, etc.):** Data structures that specify the basic structure of figures.
- **IK Solver (FABRIKSolver):** The main implementation of the Inverse Kinematics method.
- **IO Managers (SaveManager, ExportManager):** Manage export logic and save/load functions when working with model data.
- **Helper Classes:** Point, LimbAngles, KeyframeSegment, Utility, etc.

View: In charge of showing the user interface and presenting the data from the model. Business logic shouldn't be in it. Important elements consist of:

- **MenuView:** The scene of the main program window that includes the timeline view, toolbar, canvas area, and menu bar. creates the general user interface design. The canvas is used to render the stick figures using the GraphicsContext.
- **TimelineView:** The intricate user interface element that shows the play head, keyframe/interpolated cells, layers, and frame numbers.
- **FrameCell:** A unique user interface component that shows a single frame on the TimelineView.
- **Dialogs:** E.g. ExportProgressDialog, a window to show the progress of the current export.
- **GameWindow:** The primary JavaFX scene management and stage-building tool.

Controller: Serves as a bridge connecting the View and the Model. It processes user input from the View, converts it into Model actions, and refreshes the View in response to Model modifications.

- **MenuController:** The main coordinator. It oversees the synchronization of the overall state of the program, handles menu actions, toolbar interactions (such as tool changes and onion skinning), save/load/export requests, and implements AnimationUpdateListener to get updates from the TimelineModel. The other controllers are wired and owned by it.
- **CanvasController:** Manages specific user interactions with the drawing canvas, such as point dragging, figure translation in MOVE mode, and canvas context menus. To

change figure poses, it interacts with the FigureManager and notifies the MenuController of necessary adjustments (such as selection).

- **TimelineController:** Oversees user actions sent from the TimelineView user interface, such as keyframe drag-and-drop, frame cell clicks/context menus, playhead dragging, and button clicks. The TimelineModel is altered in response to these interactions.

Example of Interaction Flow (adding a keyframe with a button):

1. The user clicks the "Add Keyframe" button in the TimelineView.
2. TimelineView's button setOnAction handler calls TimelineController's handleAddKeyframeClicked().
3. TimelineController iterates through all the FigureIds by retrieving it through the TimelineModel's getManagedFigureIds(). For each figure without a keyframe at the current frame (retrieved with the model's getCurrentFrame()), it calls the model's addKeyframeData(figureId, frame).
4. TimelineModel's addKeyframeData(...) determines it needs the current pose. It calls MenuController's requestFigureStates() and MenuController's requestFigureAngles().
5. MenuController receives the requests and calls figureManager's getAllFigureStates() and figureManager's getAllFigureAngles(), returning the data.
6. TimelineModel receives the state data, stores it in its internal keyframe maps (figureKeyframes, figureKeyframeAngles), updates related interpolation, and calls notifyStateChange().
7. TimelineModel's notifyStateChange calls MenuController's onTimelineStateChanged().
8. MenuController's onTimelineStateChanged schedules requestTimelineUIUpdate() on the JavaFX thread.
9. MenuController's requestTimelineUIUpdate calls timelineView's updateUIFromEngine().
10. TimelineView's updateUIFromEngine reads the latest state from TimelineModel (including the newly added keyframe) and updates the appearance of the relevant FrameCells.

The code structure is improved by this MVC division, which guarantees that user action handling, data administration, and UI logic are separate. **Figure 3** is a high-level abstract diagram representing the MVC architecture, including the different packages and classes

from the software.

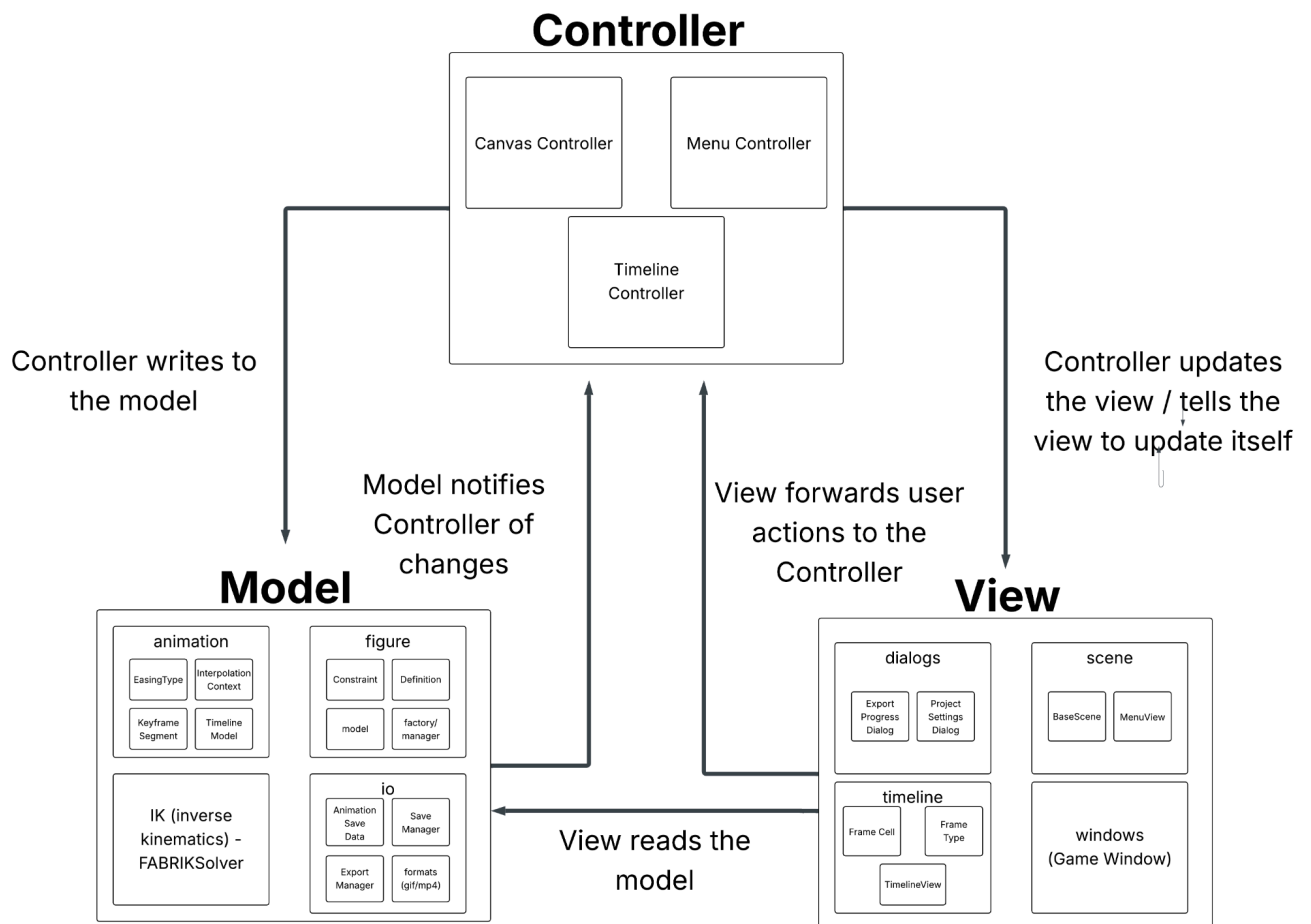


Figure 3: MVC architecture

4.4 Core Component Implementation

Figure Representation (model.figure): FigureDefinition and related classes are used to define figures structurally. These definitions are transformed into runtime StickFigureModel objects by the StickFigureFactory. Lists of Segments (linking points), IKChains (defining limbs for IK), PointData (position, appearance, drag state), and Constraints (maintaining fixed distances) are stored in each StickFigureModel. The FigureManager maintains the collection of all StickFigureModel instances.

Timeline Engine (model.animation.TimelineModel): This class serves as the main location to store animation data. It stores keyframe poses and angles per figure per frame using structures like Map, and others. In order to specify interpolation ranges and kinds (EasingType), it also controls KeyframeSegment objects. Using keyframe data and easing functions, the performInterpolation method computes intermediate frames, which are then stored in distinct maps (figureInterpolatedFrames, figureInterpolatedAngles). It requests information and communicates state changes using the AnimationUpdateListener interface.

View Rendering (view.scene.MenuView, Canvas): For drawing, the MenuView makes use of a JavaFX Canvas. The FigureManager provides the figures, which are iterated through the redrawCanvas method (called by the MenuController). It uses the GraphicsContext to draw segments and points after retrieving the current pose (either straight from the StickFigureModel or overwritten by information from the TimelineModel using the listener). To do onion skinning, the TimelineModel's past and future keyframe poses are retrieved (via MenuController) and transparently drawn before the main figure is drawn.

Timeline UI (view.timeline.TimelineView): This view handles the display of several figure layers. UI rows (a HBox with a Label and FrameCells) are dynamically added or removed according to TimelineModel-managed figures. It visually represents the frame status (empty, keyframe, interpolated) using FrameCell components. It manages vertical scrolling for layers (using ScrollPane) and horizontal scrolling for frames. The playhead on the timeline can be dragged left and right through the TimelineController.

Controller Logic (controller package): Controllers (MenuController, CanvasController, TimelineController) manage particular input areas (menus/toolbar, canvas, timeline), and convert user actions into model updates or display modifications.

Input/Output & Export (model.io package): The SaveManager handles data persistence for loading and saving animation projects. It serialises and deserializes the application state to/from JSON format using the Jackson Databind library. Rendering animation frames and encoding them into various output formats are coordinated by the ExportManager. Additionally, the GifSequenceWriter class, which adapts code originally created by Elliot Kroo under a Creative Commons Attribution 3.0 license, is necessary for the export capabilities for animated GIFs. The details of creating GIF information for looping and frame delays are handled by this modified code. The JavaCV library provides an interface to the FFmpeg video encoding toolkit for exporting to MP4 format.

4.5 Inverse Kinematics (FABRIK implementation)

Inverse Kinematics (IK) was used to enable intuitive posing by dragging hands or feet.

Problem: Determine the appropriate positions (or angles) of the connecting joints (e.g., elbow) to reach the target while maintaining the segment lengths and the base position of the chain, given a desired target position for the endpoint of a kinematic chain (e.g., a hand) and fixed lengths of the chain's segments (e.g., upper arm, lower arm).

Algorithm Choice: FABRIK (Forward And Backward Reaching Inverse Kinematics): For two main reasons, FABRIK was selected over alternative techniques (such as Jacobian-based approaches):

- **Simplicity:** Its basic idea of pulling and pushing joints iteratively is quite simple to comprehend and put into practice.

- **Efficiency:** FABRIK is computationally cheap and converges rapidly for the basic 2-segment chains (upper/lower arms and legs) employed in the stick figure.
- **Implementation (model.ik.FABRIKSolver):**
 - The static solve method takes the current joint positions, segment lengths, the target position, tolerance, and a maximum iteration count.
 - **Reachability Check:** It first checks if the target is reachable (distance from base $\leq$ total chain length). If not, it stretches the chain straight towards the target and returns false.
 - **Iteration:** If reachable, it enters a loop that continues until the distance between the end effector and the target is within the tolerance or maxIterations is reached.
 - **Backward Pass:** Inside the loop, it first snaps the end effector position directly to the target. Then, starting from the joint *before* the end effector, it iterates backwards towards the base. Each joint $P[i]$ is repositioned along the line connecting it to the next joint $P[i+1]$ (which was just moved) such that the distance between them equals the required $segmentLength[i]$. (Helper `repositionPoint` is used).
 - **Forward Pass:** After the backwards pass, the base joint $P[0]$ is reset to its original fixed position. Then, starting from the base, it iterates forward towards the end effector. Each joint $P[i+1]$ is repositioned along the line connecting it to the previous joint $P[i]$ (which was just moved) such that the distance equals $segmentLength[i]$.
 - **Convergence Check:** The distance between the end effector position and the target is checked again.
 - The method returns true if the target is reached within tolerance.
- **Integration:** The FABRIKSolver is used in the `solveSingleIK` method of the `StickFigureModel`. Usually, when an end-effector point or a middle joint is moved in HAND tool mode, the `FigureManager` (which is called by the `CanvasController`) calls the method. After the IK is solved, constraints are applied iteratively to make sure that additional geometric relationships are preserved (for the joints outside the chain that was solved with IK). The `reconstructLimbFromAngles` method skips IK and positions limbs directly on interpolated `LimbAngles`, allowing for seamless playback between keyframes with saved angles.

4.6 Visual Presentation

This section contains screenshots showing the main editor window, the timeline with numerous layers and keyframes, figure manipulation on the canvas, and the export dialogs, to present the application's user interface.

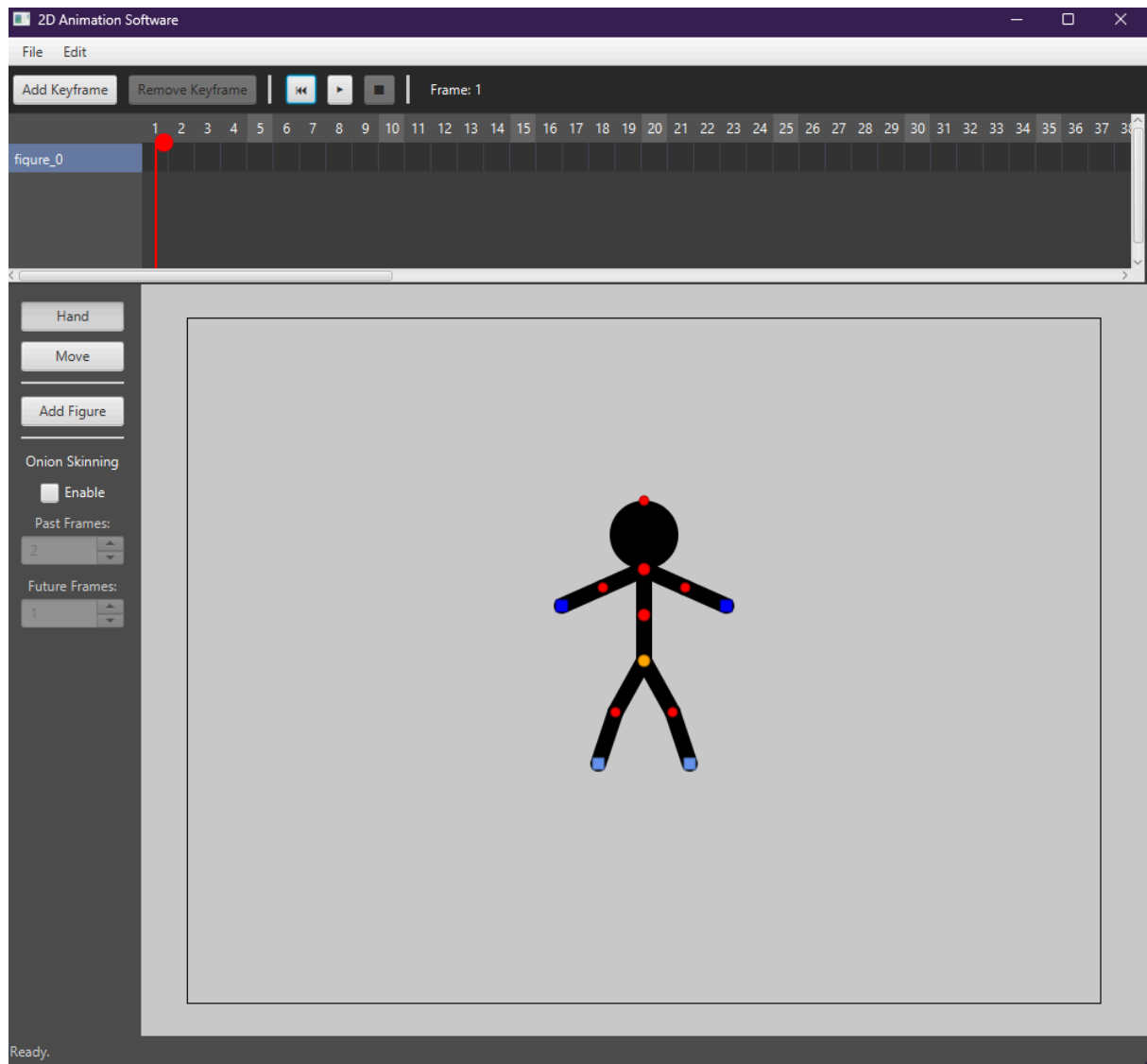


Figure 4.1: Main application window showing figure, timeline, and the toolbars.

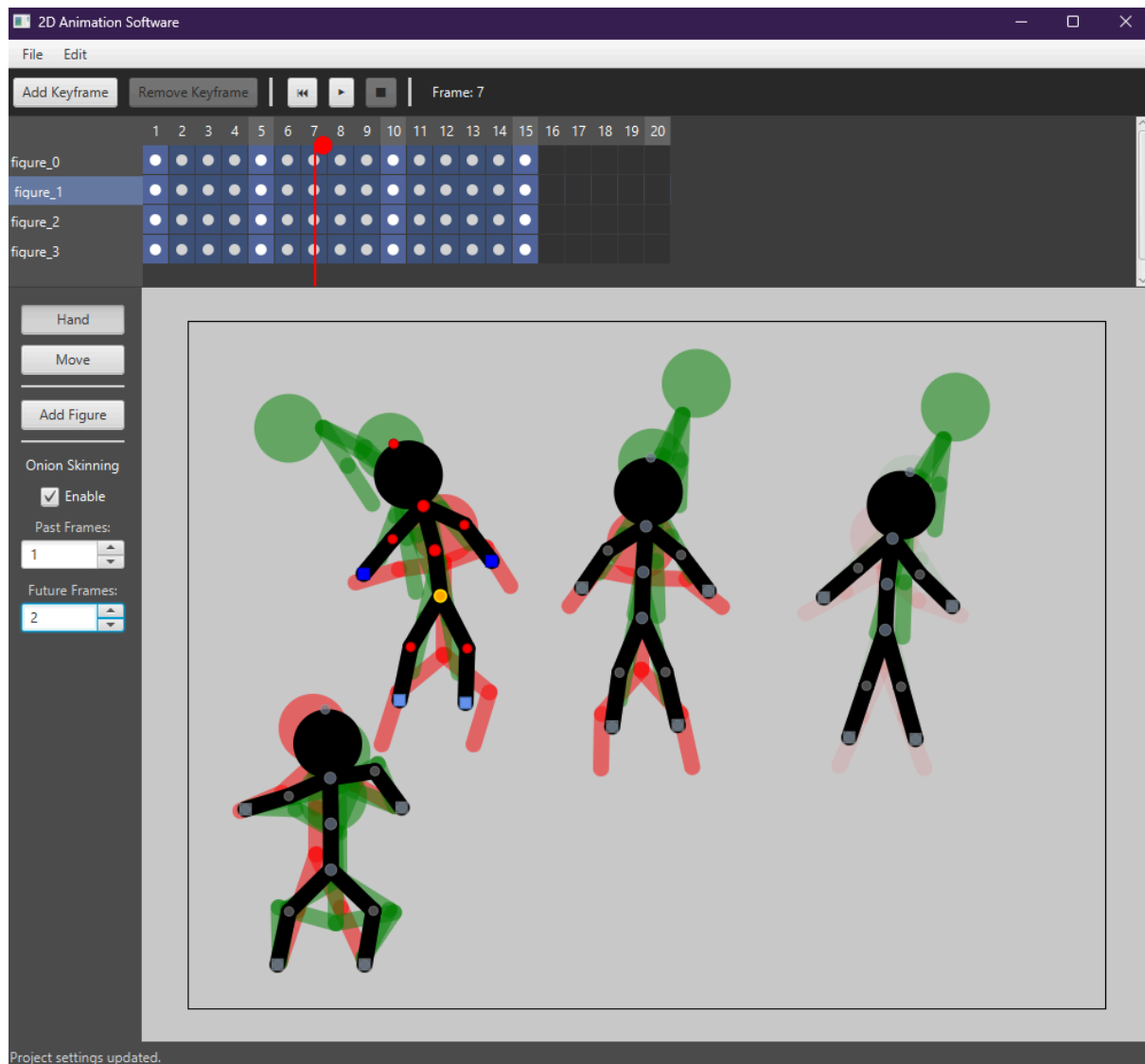


Figure 4.2: Timeline detail showing keyframes, interpolation, onion skinning and multiple layers.

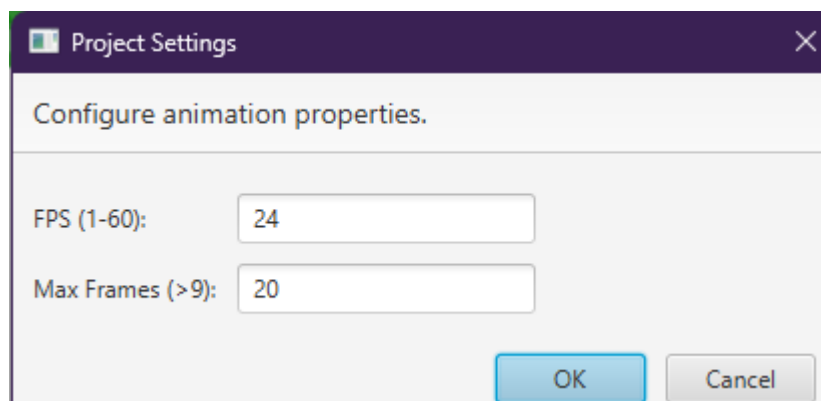


Figure 4.3: Project settings dialog accessed from Edit button at the top.

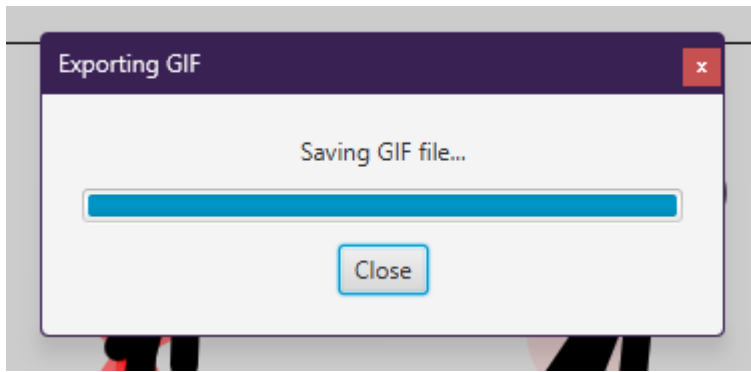


Figure 4.4: Export dialog example 1: Exporting in GIF format (during exporting).

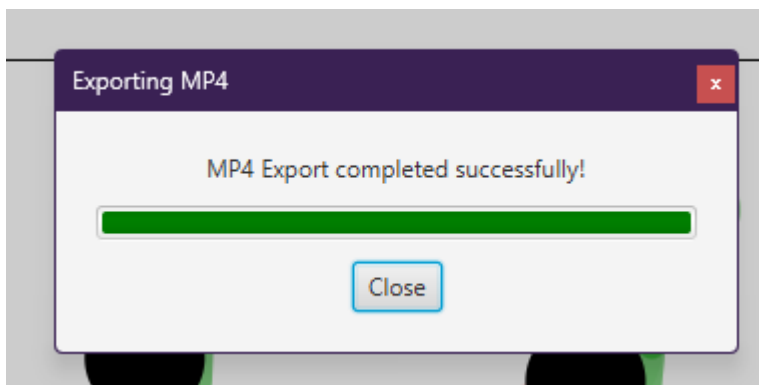


Figure 4.5: Export dialog example 2: Exporting in MP4 format (Once completed).

4.7 Implementation Summary

The implementation uses the FABRIK algorithm for efficient Inverse Kinematics, adheres to the MVC pattern for architectural structure, and successfully incorporates JavaFX for the user interface. With the help of the selected technology stack, important functionality like keyframing, interpolation, multi-layer editing, save/load, and export has been made possible. Understanding and possible future additions are made simpler by the division into Model, View, and Controller components.

5 Testing

Throughout the development lifecycle, a variety of testing methods were needed to make sure the 2D Animation Package worked properly and achieved its intended objectives. This section describes the methods, such as black-box testing against the specified requirements and white-box testing during development, and talks about the results.

5.1 Testing Methodology

There were two main methods of testing used:

White-Box Testing: This method concentrates on the application's internal organization and coding logic. It entails evaluating distinct parts, operations, and how they interact to make sure that, from the standpoint of a programmer, they operate as intended. Throughout the development phase, this was done continuously.

Black-Box Testing: With this approach, the functionality of the program is tested according to its specifications without revealing how the internal code is implemented. As a "black box," it accepts inputs in the form of user actions and compares the software's behaviour, visual results, and produced files to the requirements specification's intended results. This was done at the end of the development phase.

5.2 White-Box Testing Procedures

Several white-box approaches were often employed as work went on:

Incremental Testing: As features were put into use, they were tested. For example, before fully integrating the IK solution into the StickFigureModel, I tested its accuracy using varying chain lengths after creating the FABRIKSolver.

Logging: Much use was made of the Log4j2 framework to track the execution flow, variable states, and the underlying reason of unusual behavior during runtime, debug logs were introduced to the relevant parts of the code. This was quite helpful in identifying different problems and debugging issues.

Debugging: Code execution was often stepped through line by line using the debugger in the Integrated Development Environment (IDE). This made it possible to examine variable values at particular breakpoints, which was especially useful for identifying intricate relationships between the MVC components.

Code Review (Self-Review): Even though this solo project did not include formal peer reviews, a thorough self-evaluation of the code logic helped find potential errors, edge cases, or areas for simplification before they became serious problems.

This continuous white-box approach enabled early detection and correction of issues within individual components, resulting in a more stable foundation for future black-box testing.

5.3 Black-Box Testing (Requirements Verification)

Carefully confirming each of the functional and non-functional requirements, which was explained in **Section 3.1**, was the main goal of the black-box testing approach. A particular test condition or user activity was specified for every requirement, along with the anticipated result. The action was then carried out by the software, and the actual outcome was noted and contrasted with the prediction. Finally, the requirements were then finished off with either a fail or a pass.

Appendix 4 contains the complete Requirements Assessment Table, which includes information on each requirement, its MoSCoW priority, the test condition, the expected and actual results, and the pass/fail status.

All things considered, the application fulfilled all of the "Must Have" and most of the "Should Have" requirements. However, a number of requirements were either not implemented or only partially addressed, mostly as a result of prioritisation and time restrictions. **Table 3** below provides a summary of the conditions that were either partially completed or failed, and they will be reviewed later.

Table 3:

Req ID	Category	Requirement Description	MoSCoW	Test Condition/Action	Expected Result	Actual Result	Status
F2.e	2D Animation Creation	Add features like a path-based interpolation editor.	Should	Attempt to find path editing tools for interpolation.	UI element exists to define custom motion paths.	Only predefined easing (Linear, Quad In/Out/InOut) available via context menu.	Fail
F2.g	2D Animation	3D Camera	Could	Look for camera controls in the	Ability to rotate/pan	The UI is strictly 2D.	Fail

	Creation	for easy view.		UI.	view in a 3D-like manner.		
F4.b	Exporting Animations	Include settings for export quality, frame rate.	Could	Check Export dialogs or Project Settings for quality options.	Options available to set resolution/quality/FPS.	Export uses current canvas size and project FPS, which is editable in Project Settings.	Partial Pass
F5.b	Physics System	Provide a toggle to enable/disable physics.	Could	Look for a physics toggle in UI.	A checkbox/button exists to control physics.	No physics system implemented.	Fail
F6.a	Customization	Allow users to adjust skeletons/create custom figures.	Should	Attempt to find a figure editor or skeleton modification UI.	UI exists to add/remove/resize points/segments.	Only pre-built stick figures can be added; no editor provided.	Fail
NF4.a	Accessibility	Options to customize UI (font size, dark mode).	Could	Explore settings/menus for UI customization.	Options exist to change theme/fonts.	UI uses a fixed dark theme; no customization options.	Fail

5.4 Discussion of Testing Results

According to **Appendix 4**, all of the core "Must Have" requirements were successfully met: The GUI layout (F1.a), Posing stick figures (F2.a), basic keyframing (F2.b), multi-figure support (F2.c), real-time playback (F2.d), IK implementation (F3.a), essential export formats (F4.a), basic performance (NF1.a), IK responsiveness (NF1.b), basic usability (NF2.a), and auto-save reliability (NF3.a).

Several "Could Have" and "Should Have" conditions were either partially or not at all fulfilled:

Failed "Should Haves":

- **Path-Based Interpolation (F2.e):** It was determined that the project schedule was too tight to implement a visual path editor for motion curves, such as Bezier curves. Although the current implementation provides basic non-linear motion with four standard easing functions (Linear, Ease-In-Quad, Ease-Out-Quad, and Ease-In-Out-Quad), it lacks the fine-grained control of a path editor.
- **Custom Figure Editor (F6.a):** The project's scope and complexity would have been greatly extended if a specialized editor had been built to generate or alter figures (adding/removing points and segments). It was decided to concentrate on employing a single, pre-defined stick figure structure produced by the StickFigureFactory in order to guarantee the fundamental animation and IK capabilities were reliable.

Failed "Could Haves":

- **Accessibility Options (NF4.a, NF4.b), Physics System (F5.a, F5.b), and 3D Camera (F2.g):** These lower-priority features were not tried because of the time needed to create and test the basic animation, IK, timeline, and export functionalities ("Must" and "Should" criteria). It would take a significant amount of development work to handle complicated interactions and integrate a separate engine for physics in particular.

Partially Met/Failed Requirements:

- **Export Settings (F4.b - Could):** Although MP4 and GIF export operates, time constraints prevented the addition of UI controls to modify the resolution, quality presets (for MP4), or frame delays (for GIF). These changes required a greater involvement with the underlying libraries (JavaCV/FFmpeg, ImageIO) and additional UI design. Currently, the export makes use of the canvas's current dimensions (not editable) and the project's frame rate (editable in the project settings).

5.5 Testing Summary

The testing methods verified that the 2D Animation Package effectively implements its essential features by combining systematic black-box testing against requirements with ongoing white-box tests during development. IK posing, keyframing, interpolation, multi-figure timelines, save/load, and common output formats are among the functional characteristics needed for simple stick figure animation. Due to time and resource constraints that come with the project scope, features that were deemed lesser priority ("Could Haves") or more difficult "Should Haves" (custom figure editor, path interpolation) were left out. In

general, the functions that have been added meet the usability requirements for a simple animation tool and operate with stability.

6 Evaluation

A qualitative study was carried out with some users to evaluate the resulting 2D Animation Package's usability, intuitiveness, and efficiency, especially when compared to other entry-level tools like Pivot Animator. Since the participants had never used animation software before, they were required to do simple animation tasks using Pivot Animator and the built program. Semi-structured interviews and observations were used to collect their opinions and experiences, with particular attention paid to the influence of Inverse Kinematics (IK), simplicity of use, and desired features or enhancements.

The main themes that surfaced from the participant responses are covered in the sections that follow.

6.1 Inverse Kinematics (IK) as a Core Strength

One recurring topic was how the integrated Inverse Kinematics technology improved the animation workflow, particularly when it came to creating postures that looked natural. Compared to Pivot's joint-by-joint Forward Kinematics (FK) method, participants consistently reported that posing the figure using IK was much simpler and more straightforward.

The IK approach, according to the participants, allowed for more fluid and lifelike movements. According to Participant 3 (P3), "if you wanted something that looked more realistic, then this one [the developed software] is actually better because the way you can move the limbs can look more natural... This can be used for more complex body movements, it's better." This suggests that IK was successful in reducing the difficulty of producing realistic poses.

When compared to changing each joint separately, having the ability to control complete limbs by moving an end-effector (hand or foot) was seen to be a significant time-saver. P1 said, "I'm gonna have to say your one [is better] purely because I can move [the limbs] all in one go, which makes it easier than moving every single joint." Identically, P2 said, "It did a lot of animating for you, I can just kind of move it in the direction I want to go in, and movement was more natural so I didn't have to do much for each frame."

The IK system generally acted as participants had anticipated. P2 said, "It feels fairly correct, nothing too unexpected [happened while posing] really," while P3 stated, "I don't think anything that happened which I WASN'T expecting". This suggests that the FABRIK

implementation for inverse kinematics was sufficient since it produced results for these typical limb movements that were predictable and clear.

Although generally favourable, a few participants pointed out possible slight disadvantages for extremely precise but basic motions. P3 mentioned, "It's harder to do 'simple looking animation' but easier for more complex. Like if I wanted to do a cool pose, it's a lot easier with your tool but for a simple limb raising, then it's a little harder." P2 said, "I guess if you wanna do a wave it may be harder, maybe if it's something very specific with the hand that you want to change, it can make it harder". This demonstrates that although IK is excellent for posing in general, direct FK control may occasionally be better for isolated, very basic joint rotations.

Overall, the feedback strongly supports the project's goal of using IK to make character posing more intuitive and efficient, especially for users who have no prior animation experience.

6.2 Timeline Management and Workflow

The idea of giving each figure its own timeline row was comfortably accepted. P1 commented, "It's also nice the timelines are separate... compared to Pivot. It would be easier to fix a mess-up on a figure. Definitely prefer it being separate. Gives you more control on every little detail." This supports the design objective of enabling individual control over several characters.

Compared to Pivot, several participants found the keyframing process to be a little less intuitive because my tool's timeline doesn't progress instantly when you press "Add Keyframe." P2 pointed out that "The frame doesn't jump to the next one after I pressed 'add keyframe', which was confusing because in pivot it did." P3 also found adding keyframes "harder" when compared to Pivot.

One clear benefit was being able to look at the interpolated poses. P1 said, "With your software, I can look at each interpolated frame and how it turns out, in Pivot the interpolated frames are only green lines between the keyframes so you can't actually see them." This visual feedback is essential for understanding the resulting motion.

The timeline's layer structure works well, but usability might be further improved by simplifying the keyframing process (possibly by including an option for automated frame advance).

6.3 User Interface (UI) and Intuitiveness

Compared to Pivot, the user interface was thought to be more straightforward. According to P2, "I feel like your one is more explicit" in comparison to Pivot, where you had to hover over symbols to understand what the various buttons did.

P1 remarked, "maybe the little area of pressing should be slightly bigger, it was hard to click on the joints sometimes." This feedback can be incorporated easily by simply increasing the hit-box size for draggable locations.

P2 proposed enhancing discoverability for actions such as removing figures: "Maybe it would help if you could right-click on the figure instead of just the points. When you're moving or deleting." At the moment, context menus are connected to the stick figure's joints or timeline layers. It could be useful to have context menus on the stick figure itself (the segments).

Although the user interface is feasible for amateurs, minor improvements such as bigger selection targets and possibly more extensive context menu access could improve the user experience.

6.4 Missing Features and Future Improvements

An important drawback in contrast to Pivot was the inability to modify the stick figure's structure. As P1 pointed out, "You can't edit the figure, unlike in Pivot. It's tougher to have creative characters or customization." This identifies a critical area for further development and has a direct connection to the failed "Should Have" criteria (F6.a).

Several participants identified the absence of an undo capability as a critical component that is necessary for learning from mistakes. P2 made the point that "there isn't an undo tool which makes the program difficult to use if you make a mistake," and P3 echoed, "Maybe an undo feature would have been nice if I accidentally messed up the entire pose." This is a high-priority addition and a usual expectation in creative software. It was overlooked at the design stage even though it should have been a requirement.

Mirroring tools (P3: "Well, do you have a mirroring tool? "), as well as backdrop images and colours (P2: "Maybe a background image... It might help if you could change the background colour or something") were among the suggestions. It would help with animation. These would improve workflow efficiency and creative possibilities.

Beyond standard interpolation, P1 expressed a desire for direct frame-by-frame control with interpolated frames ("It would be nice to edit the interpolated frames"). This suggests that P1 may have wanted an interpolation editing system to make the interpolated frames more precise to what they wanted (referring back to F2.e) or more direct frame-by-frame control.

P3 also mentioned that they would want previews ("The previews of the keyframes would be useful... Makes it easier to animate"), which prompts reflection about the timeline design that was selected. The original concept (**Section 3.2**) used abstract cells to represent frames in a timeline format comparable to that of a program like OpenToonz. Despite being common in many animation workflows, P3's comments, which were probably influenced by the evaluation's direct comparison with Pivot Animator's thumbnail-based timeline, indicate that for complete beginners, Pivot's method of displaying a brief visual preview of the pose within the timeline might be more instantly understandable and useful for monitoring animation flow. I may have to reevaluate the idea that the OpenToonz approach was superior or simpler for this particular target demographic. This implies that providing visual previews might be a worthwhile option for future UI development targeted at beginners.

All of the user feedback helps to give distinct directions for further research. To close significant functional gaps, a strong undo system must be implemented first, followed by the addition of figure customization features. The software's usability and appeal would then be greatly increased by adding visual elements (backgrounds, mirroring tools), making interpolation more clear and interactive (addressing P1's feedback), and possibly reexamining the timeline's visual representation (addressing P3's feedback and the design reflection). This would especially benefit the software's target audience of inexperienced animators.

6.5 Evaluation Summary

When compared to more conventional FK-based tools like Pivot Animator, the user review shows that the 2D Animation Package effectively used Inverse Kinematics to deliver a simple and effective posing experience, which is especially valued by new users. For scenarios with multiple characters, the distinct timeline layers also provide more control. However, the evaluation also pointed out important areas that needed work. The two biggest functional limitations found were the absence of an undo mechanism and the inability to customize figures. Small usability problems were identified, such as workflow stages (keyframe advancement) and point selection accuracy. Although the underlying IK and animation system is robust, the software's effectiveness and user enjoyment would be greatly increased by incorporating these missing functions and improving the user interface/workflow, bringing it closer to bridging the gap between basic free tools and more sophisticated professional software.

7 Project Management

This section describes the planning, risk assessment, tools, management strategy, and reflections associated with the creation of the 2D Animation Package.

7.1 Development Approach and Methodology

As originally stated in the progress report, an iterative development approach was used for this project. The inclusion of testing throughout the development lifecycle and gradual progress were all made possible by this methodology. The fundamental idea was to create a Minimum Viable Product (MVP) with the required features initially, and then iteratively make improvements and refinements that aligned with the requirements.

Phases of development were grouped according to the MoSCoW plan:

Phase 1 (core features development) included all the **MUST-have** requirements, to ensure that all the essential features are in place and provide a solid foundation.

Phase 2 (additional features development) included the **SHOULD-have** requirements, to finalise the product.

Phase 3 (polishing and advanced features) included any remaining **SHOULD-have** requirements and **COULD-have** requirements. This phase was significantly shorter due to time constraints, leading to the omission of most "Could Have" features and some "Should Haves".

By providing us flexibility in managing the schedule by adjusting to obstacles, this iterative process made sure that the most important functionality was delivered.

7.2 Project Management Tools

To efficiently manage the project, several tools were used:

Initially, a Gantt chart was made to show the project's tasks and deadlines (see **Section 7.4**). This functioned as an overview of the project's phases and due dates.

During development, Notepad++ was used to record ideas, take brief notes, and keep track of little errors or tasks to be done. During quick note-taking sessions, its ability to recall unsaved changes upon closure was especially helpful in avoiding data loss.

The supervisor's online booking system enabled the scheduling and execution of regular supervision meetings over Zoom. These gatherings were essential for talking about developments, getting input, and resolving possible obstacles.

Instead of using a formal version control system like Git, a manual method was used. The whole project directory was manually backed up into older folders before beginning any potentially disruptive changes or major new features. This reduced some of the risks by offering a safety net to roll back to a previous working state if significant problems developed.

7.3 Risk Analysis and Mitigation

To find potential obstacles, a risk analysis was carried out at the start of this project (**Appendix 5**). During the project, a number of the risks that were identified came to pass, and mitigation techniques were used:

Risk 2 (underestimating time) and Risk 6 (personal limitations): These were significant considerations. The mitigation involved focusing efforts on "Must" and high-priority "Should" requirements while closely following the MoSCoW prioritization. Features like physics (Could) and the custom figure editor (Should) were purposefully postponed to guarantee that the main product would be delivered in the allotted time. It was easier to control expectations about the achievable scope when there was regular communication with the supervisor. The impact of these risks is shown in the Gantt charts (**Section 7.4**).

Risk 3 (Software Bugs): It was inevitable that unexpected bugs would appear, especially when integrating several components (e.g., TimelineModel updates correctly triggering TimelineView redraws). Manual backups proved to be an effective mitigation technique on multiple occasions, enabling a rollback if a change created a difficult-to-diagnose problem. The main methods for finding and resolving these issues were regular logging and debugging (white-box testing). For this one-off project, the manual backups were useful even though they weren't as reliable as Git.

Risk 4 (Hardware Limitations): During development, testing was done on both desktop and laptop computers as part of mitigation for this risk. The application continued to work on standard consumer hardware, therefore this risk did not significantly impact the project's basic objectives.

Risk 5 (Scope Creep): The primary mitigating strategy was the explicit MoSCoW prioritizing that was set during the design stage. Although ideas for new functionality surfaced during development, feature additions that would have threatened the core timeframe were avoided by sticking to the established requirements list.

The risk analysis helped foresee any problems. The risks that were identified helped determine priorities and emphasized the significance of mitigation techniques like backups and focusing on essential requirements when time constraints developed, even though not all preventative measures were rigorously followed (such as official Git usage).

7.4 Project Planning and Scheduling (Gantt Charts)

To map out the full project lifecycle from conception to final submission, a Gantt chart was created as part of the initial planning phase. Specific time blocks were allotted for research,

design, development phases, testing, and report writing in this initial plan. **Figure 5.1** shows the original Gantt Chart.

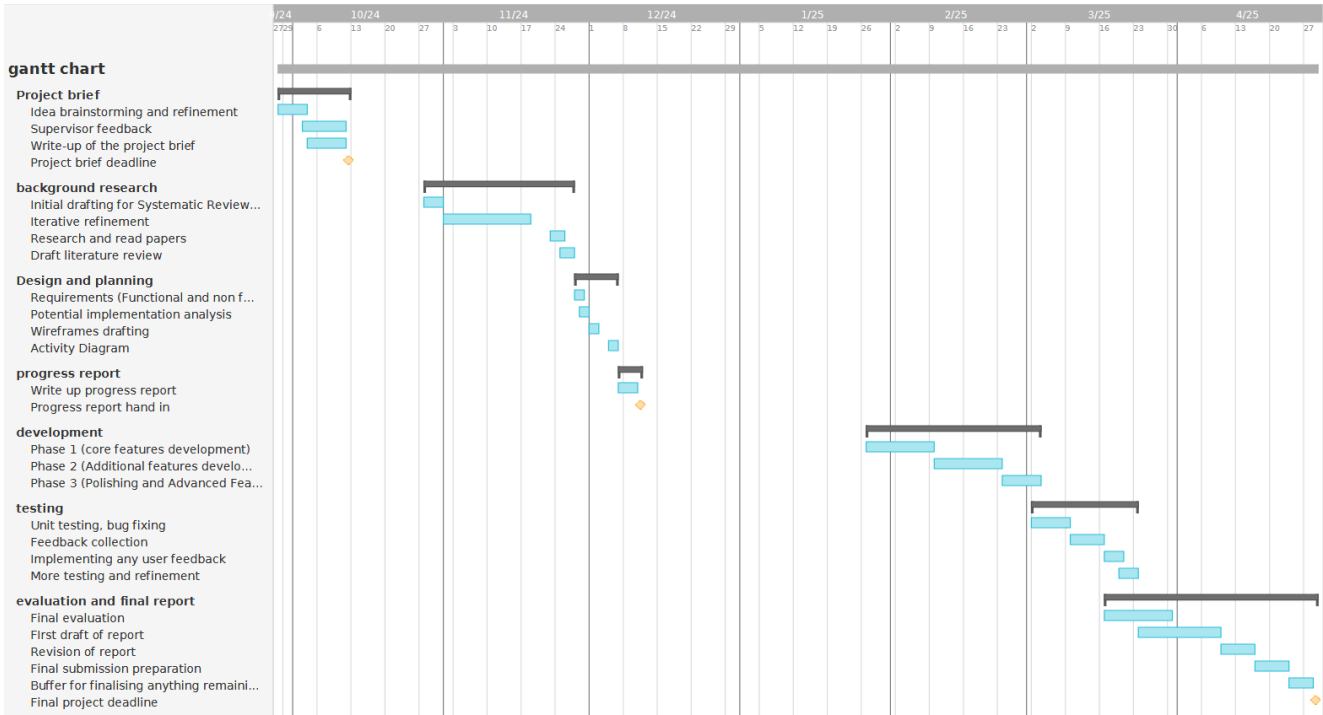


Figure 5.1: Old Gantt Chart

However, due to factors mentioned in the risk analysis (See **Section 7.3**), the actual execution differed from this initial plan, especially throughout the development and testing phases. Testing was no longer a separate step after development but rather became more integrated with it, and the development length essentially changed and shrank. For the sake of comparison, a new Gantt chart (**Figure 5.2**) was made which shows the actual time taken.

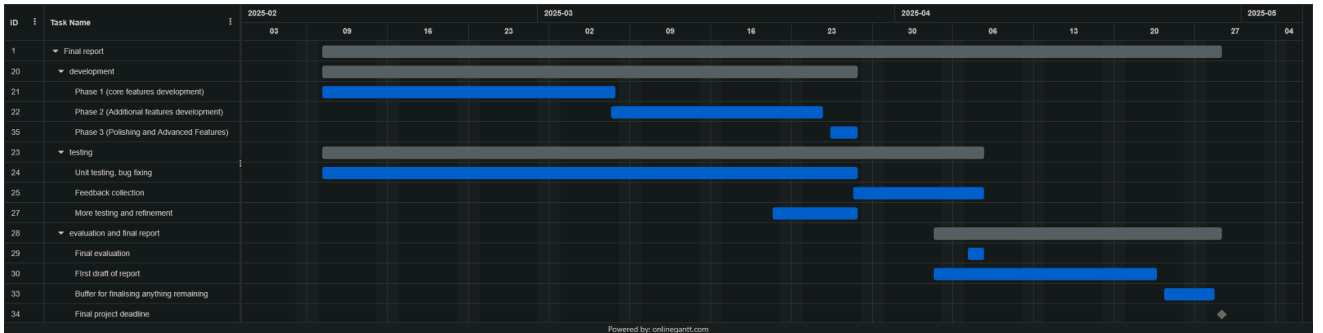


Figure 5.2: New Gantt Chart

The development phase began later than anticipated and took up a large portion of the initially specified testing block. The main causes of this were Risk 2 (**time underestimation**) for integrating intricate features like the timeline view and guaranteeing IK stability, and Risk 6 (**personal constraints**), where time had to be allotted to other university modules.

Instead of happening one after the other, testing (white-box testing) happened simultaneously with development. This helped identify flaws early and was a useful improvement to follow

more closely to the iterative process (Mitigation for Risk 3). In the second Gantt chart, the user input collection phase (for evaluation **section 6**) is represented by the "feedback collection" block.

The final development process was drastically shortened and thus most "Could Have" requirements and several "Should Haves" were not implemented because the focus switched to making sure core functionality was stable.

7.5 Personal Reflection

Despite time restrictions, a functional product was delivered thanks to the iterative process and MoSCoW prioritizing. The first Gantt chart offered a useful, though optimistic, road map. The difference between the new and old Gantt charts made it clear how crucial it is to include in buffer time and be ready to modify the scope in response to actual hazards, such as time underestimation and conflicting priorities.

For future work, there are a few things I would do differently. Even if the manual backups were successful, implementing Git from the beginning would have offered much more precise version control and simpler branching for experimental features which might have sped up debugging. Although feedback was gathered for the final evaluation, Peer involvement earlier - possibly even during the design phase (e.g. trying Pivot before finalizing requirements) - could have identified important lacks like the "Undo" functionality sooner, possibly allowing it to be prioritized within the "Must Have" category. A more accurate initial prediction could have happened if the original Gantt chart had addressed the possibility of overlap between development and testing.

8 Conclusion

This last section lists the project's achievements, recognizes its shortcomings, and suggests possible future paths for the 2D Animation Package's development.

8.1 Summary of Achievements

The goal of this project was to bridge the gap between complex, costly professional software and basic animation solutions that lacked advanced features. The main objective was to create a user-friendly and easily available 2D animation tool that would use Inverse Kinematics (IK) to enable users, especially novices and hobbyists, to produce stick figure animations that looked natural.

By using Java and JavaFX to create a working desktop application that followed the Model-View-Controller (MVC) architecture, this goal was mostly accomplished. The software that is produced enables users to:

- Create animations using pre-built stick figures.
- Utilize an intuitive timeline with support for multiple figure layers, enabling independent animation control.
- Use keyframing techniques to define poses at specific points in time.
- Have the option to use interpolation between keyframes with multiple easing options (Linear, Ease-In, Ease-Out, Ease-In-Out) to create smooth motion.
- Pose figures naturally and efficiently using the integrated FABRIK solver by simply dragging end-effectors (hands and feet).
- Visualize animation progress with features like onion skinning.
- Save and load animation projects (files saved in JSON).
- Export completed animations into formats suitable for sharing: PNG (current frame), PNG sequence, GIF, and MP4 video.

In contrast to more conventional Forward Kinematics tools like Pivot Animator, the IK implementation successfully made posing far more intuitive, as demonstrated by the user evaluations (**Section 6**). Testing verified that all important "Must Have" requirements were satisfied. The program has a robust, maintainable foundation thanks to the MVC design. The project effectively illustrates the viability of an approachable tool that uses effective animation methods such as IK.

I became more aware of the complexity needed to make an interactive program, especially when it comes to responsive user interfaces. The importance of extensive testing became abundantly clear during the process. User feedback (**Section 6**) was extremely helpful in pointing out usability problems and feature gaps that might not have been apparent during development.

8.2 Limitations

The project had several limitations, mostly related to time constraints that were faced during development and the requirement prioritization that was essential, even if the core features were implemented successfully:

- **Lack of Customization:** A significant limitation is the inability of users to create or modify the stick figures (**Failed Requirement F6.a**). Users are currently restricted to the single pre-built figure provided by the StickFigureFactory class. This limits creativity compared to tools like Pivot Animator that allow figure editing.
- **No Undo/Redo Functionality:** As highlighted strongly in the user evaluation (**Section 6.4**), the lack of an undo/redo mechanism is a major usability drawback. Accidental movements or undesired poses cannot be easily reverted, potentially frustrating users.

- **Basic Interpolation Control:** While multiple easing functions are provided, the software lacks advanced interpolation controls like a visual path-based or Bezier curve editor (**Failed Requirement F2.e**). Users cannot fine-tune the motion path between keyframes beyond selecting a predefined easing type.
- **Missing "Could Have" Features:** Several desirable but lower-priority features were not implemented, including:
 - A physics system (gravity, collisions) (**F5.a, F5.b**).
 - User-configurable export settings (resolution, quality) (**F4.b**).
 - UI customization options (themes, fonts) (**NF4.a**).
 - Enhanced UI polish and animations (**NF4.b**).
- **Evaluation Scope:** The user evaluation provided valuable qualitative insights, but the feedback was gathered from a limited number of participants. A broader evaluation with more users might reveal additional usability issues or perspectives.
- **Development Process:** The reliance on manual backups instead of a formal Version Control System like Git presented a minor risk during development, although it did not ultimately hinder the project.

8.3 Future Work

For future work, the following actions are put forward, roughly ranked by impact, based on the requirements analysis and user input:

- **Implement Undo/Redo:** Adding a robust undo/redo system is the highest priority for improving usability and user confidence. This would likely involve maintaining a history of model states, maybe with a stack data structure.
- **Develop a Figure Editor:** Creating a dedicated UI module for users to build, modify, and save custom stick figures (addressing **F6.a**) would dramatically increase the software's creative potential and bring it closer in line with user expectations set by tools like Pivot.
- **Enhance Interpolation Control:** Implementing a visual editor for motion paths (e.g., Bezier curve editor, addressing **F2.e**) would give users significantly more control over the movement between keyframes.
- **UI Refinements:** Address minor usability issues identified in the evaluation, such as increasing the selection hit-box size for points, potentially adding figure-level context menus on the canvas, and refining the keyframe workflow (e.g., option for

auto-advance when adding a keyframe). Adding UI polish (**NF4.b**) and basic customization (**NF4.a**) would also improve the user experience.

- **Add Export Settings:** Provide users with options to control resolution, and quality/compression settings during the export process (addressing **F4.b**).
- **Physics Integration:** Explore the integration of a simple, optional 2D physics engine (like Phys2D or an implementation based on Position Based Dynamics) to handle effects like gravity or basic collisions (addressing **F5.a**, **F5.b**), further enhancing realism where desired. Secondary motion can also be explored, for new types of figures like animals which may have tails that naturally move with gravity.
- **Further Enhancements:** Consider features like bone stretching/squashing, sound integration, or timeline thumbnails based on further user feedback.
- **Distribution:** Once polished and key feedback is added in (especially Undo and Figure Editing), the software could be packaged and distributed as a free tool for aspiring animators, fulfilling the project's core aim of increasing accessibility to animation tools with IK capabilities.

In conclusion, this project effectively produced a working 2D stick figure animation tool that shows how Inverse Kinematics may be used to create a user-friendly workflow, especially for inexperienced animators. The core program confirms the initial idea that effective animation methods may be made available in an approachable package, even though there were features that were left out due to scope constraints (mostly related to customisation and further advanced capabilities). The developed program offers a strong foundation for further improvements and has the potential to evolve into a useful tool that reduces the entry barrier for aspiring animators who want to experiment with character motion beyond the constraints of simple Forward Kinematics tools.

References

- [1] Wells, Paul. Animation : Genre and Authorship. Wallflower, 2002.
- [2] Maestri, George. Digital Character Animation 3. New Riders, 2006.
- [3] Sito, Tom. Moving Innovation - a History of Computer Animation. Mit Press Ltd, 2013.
- [4] "Evolution of Animation Techniques: From Traditional to Digital | RMCAD." *RMCAD*, 17 Oct. 2024, www.rmcad.edu/blog/the-evolution-of-animation-techniques-from-traditional-to-digital/. Accessed 26 Nov. 2024.
- [5] Soriano, Marc. "Skeletal Animation." *Ucr.edu*, 5 Jan. 2011, alumni.cs.ucr.edu/~sorianom/cs134_09win/lab5.htm. Accessed 9 Apr. 2025.

- [6] Kucuk, Serdar, and Zafer Bingul. "Robot Kinematics: Forward and Inverse Kinematics." *Industrial Robotics: Theory, Modelling and Control*, Dec. 2006, <https://doi.org/10.5772/5015>. Accessed 17 May 2021.
- [7] Ruuskanen, Antti. *Inverse Kinematics in Game Character Animation*. 2018, www.theseus.fi/bitstream/handle/10024/156571/Ruuskanen_Antti.pdf?sequence=1/1000. Accessed 9 Apr. 2025.
- [8] Fridsma, Lisa, and Brie Gyncild. *Adobe after Effects Classroom in a Book (2020 Release)*. Adobe Press, 2020.
- [10] Meredith, Michael, and Steve Maddock. *Real-Time Inverse Kinematics: The Return of the Jacobian*. 1 Jan. 2004..
- [11] Aristidou, Andreas, and Joan Lasenby. "FABRIK: A Fast, Iterative Solver for the Inverse Kinematics Problem." *Graphical Models*, vol. 73, no. 5, Sept. 2011, pp. 243–60, <https://doi.org/10.1016/j.gmod.2011.05.003>.
- [12] Parent, Rick. *Computer Animation: Algorithms and Techniques*. Elsevier, 2012.
- [13] Müller, Matthias, et al. "Position Based Dynamics." *Journal of Visual Communication and Image Representation*, vol. 18, no. 2, Apr. 2007, pp. 109–18, <https://doi.org/10.1016/j.jvcir.2007.01.005>. Accessed 8 Oct. 2020.
- [14] Willett, Nora S., et al. "Secondary Motion for Performed 2D Animation." *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*, Oct. 2017, <https://doi.org/10.1145/3126594.3126641>.
- [15] Castillo, Del. "Physics-Based Interpolation for Animation Using Physics-Informed Neural Networks." *Handle.net*, Nov. 2023, hdl.handle.net/1969.1/200260. Accessed 26 Feb. 2025.
- [16] Blazer, Liz. *Animated Storytelling*. Peachpit Press, 2015.

Appendices

Appendix 1: Systematic Review Descriptions

1.1 Pivot Animator

Starting with Pivot Animator, this is an incredibly simple software that anyone can pick up and start using straight away. With a canvas in the middle, tools on the left and a timeline at the top, the interface is straightforward to understand for anyone. This software is free but has a limited amount of customizability with the characters you can create. It's centred around stick figures, so the only tools for character creation are simple shapes with lines, squares and circles. You can't draw with a brush like most other animation software. The characters you create are limited to forward kinematics, which means you have to animate them by adjusting each limb at its joint. It does give you the ability to stretch the joints as you animate, though, which can lead to more expressive and exaggerated movements, adding flexibility to the animations. However, you won't find any rigging tools, layers, or effects like other software.

Pivot is ideal for beginners and hobbyists who want to create stick figure animations without complexity. There are no physics options at all in this package, so it relies completely on the user's skills for realistic or dynamic animation. There are a limited number of export formats, mostly just AVI and GIF as it's geared towards hobbyists. See **Figure 6.1** for an image of the interface of Pivot Animator.

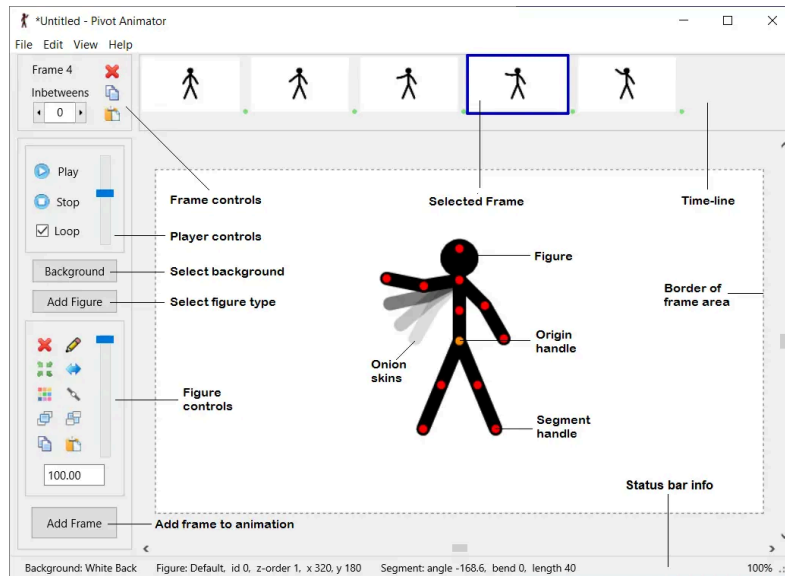


Figure 6.1: Screenshot of the Pivot Animator interface. Source: Pivot Animator official website (<https://pivotanimator.net/>)

1.2 OpenToonz

OpenToonz is another free but also open-source software. This is ideal for budget-conscious users. This software has a customizable interface, with the ability to have different workspaces. Normally, in the centre there is the canvas where you can draw and animate. The left side includes your tools and the timeline is an X-sheet on the right side that allows you to manage your animation frames vertically. You can also have the traditional horizontal timeline though. OpenToonz also supports a range of scanning tools to help integrate hand-drawn elements. Rigging is an option available in this package; however, it's more commonly used for traditionally drawn animation. Due to this, the inverse kinematics features may feel more limited than some other packages. You can assign bones to body parts and cause them to deform, whilst also having different drawings for hand poses or face angles to switch between during your animation. You can add effects to your animation in a "Stage Schematic" panel. This allows you to add effects to your layers like glow, noise, distortion, and various shaders. There is a steeper learning curve to OpenToonz compared to Pivot Animator due to the range of features but it's powerful. Like Pivot Animator though, there are minimal to no physics simulation features. OpenToonz focuses more on traditional animation techniques rather than realistic movement effects. This software is compatible with various formats like TIFF, JPEG, PSD and PNG, as well as video formats like MP4 or MOV; however, it needs additional libraries installed for this to work (FFMPEG or Quicktime). This makes it versatile for multimedia applications, although it may be more difficult to export to

video formats for social media without the correct libraries installed. See **Figure 6.2** for an image of the OpenToonz interface.

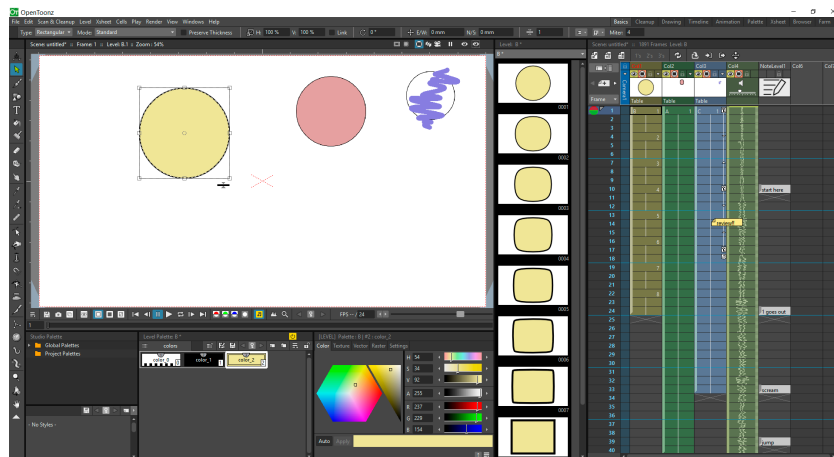


Figure 6.2: Screenshot of the OpenToonz interface. Source: OpenToonz user manual (<https://opentoonz.readthedocs.io/>)

1.3 Synfig

Synfig, like OpenToonz, is another free open-source animation package with a simple and easy-to-use interface. Synfig is designed for traditional hand-drawn animation but also provides you with tools for digital 2D animation. In the middle, there is the canvas; on the left is the toolbox with basic drawing tools such as the brush, fill tool, scale tool and many more. The tools panel also contains options for brush size and colour. On the right side, there are the panels for layers and the navigator. At the bottom (and with many animation software) is the timeline which controls the playback of the animations and motion of the characters. Synfig also has options to add filters, gradients, distortions and effects to your animation. It has a moderate amount of customization utilities. It contains a rigging system too, which lets you create bones and a virtual skeleton for characters and objects. Similar to OpenToonz, this software is catered more towards traditional animation so the IK features may seem more rudimentary, but they are present if needed by the animators for more puppet-like animation styles. Synfig is maintained by the community so there is plenty of help online. The UI is relatively easy to understand and the workflow is simple enough that the learning curve isn't as steep as some other animation packages. There are minimal to no physics features in this software though, similar to both OpenToonz and Pivott. It supports popular video formats such as AVI or MPEG, as well as GIFs and image sequences, making it useful for social media and multimedia applications. See **Figure 6.3** for an image of the Synfig interface.

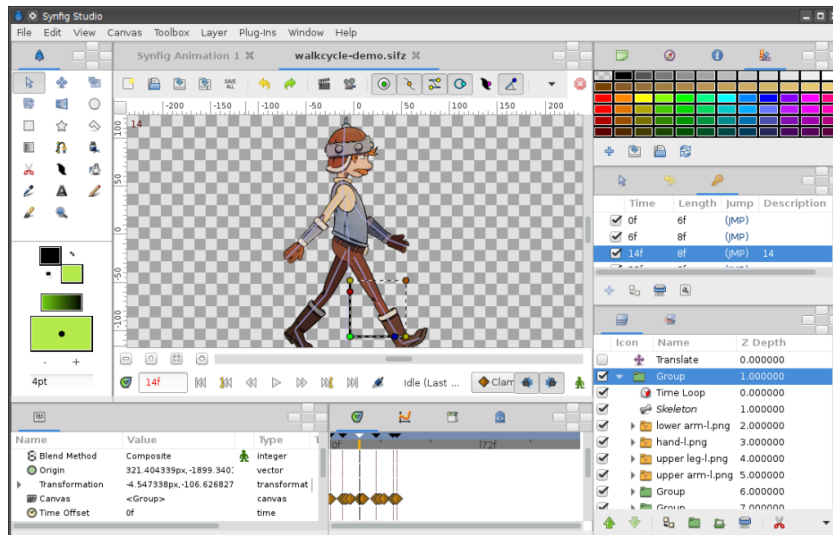


Figure 6.3: Screenshot of the Synfig interface. Source: Synfig official website (<https://www.synfig.org/>)

1.4 Moho

Moho is a paid but much more versatile animation software aimed at hobbyists and professionals. There is Moho Debut which is more affordable and for hobbyists and digital artists, while Moho Pro is more costly but has more professional features such as 3D layer and object support, real-time media connection, layer referencing, etc. The interface for Moho is customizable with the canvas in the centre, the timeline at the bottom and the tools around the workspace. Moho includes basic drawing tools but also includes a much more advanced bone rigging feature. It offers both inverse and forward kinematics. The rigging features are more robust, letting you animate your characters smoothly and more lifelike. Moho lets you create skeletal rigs which makes it well suited for cut-out animation. Being a professional-grade software, it's no surprise that it also contains a variety of effects for compositing, such as shading, blurring, and textures for depth and polish to the animation. The learning curve is much steeper than most software as it's a more advanced tool with features suited for professional use. It requires a larger time investment to master but it provides you more natural control over character movement and animation. It includes physics tools for more realistic simulations, letting you animate things that may be more difficult otherwise. Moho allows you to export to MP4, GIF and other formats like image sequences for broader casual and professional use. See **Figure 6.4** for an image of the Moho interface.

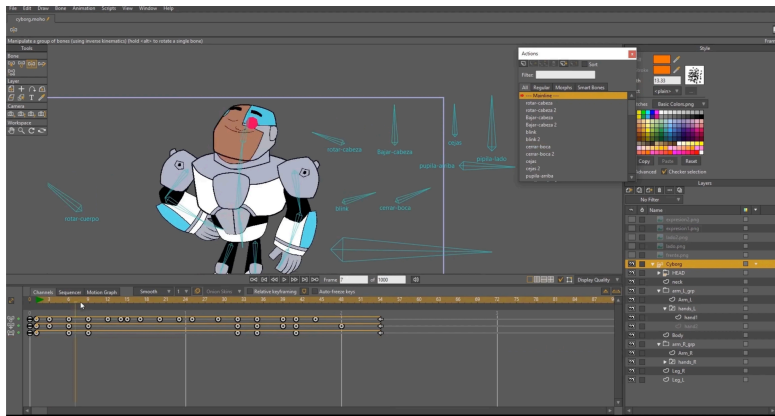


Figure 6.4: Screenshot of the Moho interface. Source: Moho official website (<https://moho.lostmarble.com/>)

Appendix 2: Functional and Non-Functional Requirements

2.1 Functional Requirements:

Table 4.1:

	Must have	Should have	Could have	Won't have
1. Interface	a. Provide a graphical user interface with the middle focused on graphics, and the sides and top used for buttons.			
2. 2D Animation Creation	a. Enable users to edit pre-built stick figures by moving the joints.	e. Add features such as path-based interpolation editor for smoother animation.	g. 3D Camera for easy view of different perspectives of the stick figures.	
	b. Ability to animate the stick figures using keyframes.			
	c. Support	f. Add onion		

	animating multiple stick figures in the same project.	skinning to preview the previous and next keyframes (relative to currency keyframe).		
	d. Enable real-time playback of animations within the software.			
3. Inverse Kinematics (IK)	a. Include a built-in IK solver (FABRIK IK) to let the user manipulate a stick figure with inverse kinematics (they must be able to move the end of a joint to position the entire joint chain).			
4. Exporting Animations	a. Support exporting animations in MP4 and GIF formats.		b. Include settings for export quality, frame rate and resolution.	
5. Physics System			a. Simulate simple effects like gravity to enhance realism.	c. Simulating forces like momentum, friction or soft-body effects.
			b. Provide a toggle to enable/disable physics per	

			element.	
6. Customization		a. Allow users to adjust the skeletons and create custom stick figures.		

2.2 Non-Functional Requirements:

Table 4.2:

	Must have	Should have	Could have	Won't have
1. Performance	a. Software must run efficiently on an average consumer hardware (e.g. 8GB ram, integrated GPU).	c. Handle projects with multiple stick figures without significant performance loss.		
	b. IK calculations should be real-time and responsive.			
2. Usability	a. Interface must be simple enough for users with no animation experience to be able to use and animate.	b. Provide visual cues for when adjusting the joints of the stick figures.		
3. Reliability	a. Auto-save functionality to avoid data loss.	b. System should not crash during common		

		operations such as saving or exporting.		
		c. Modular codebase for easier future updates or feature additions.		
4. Accessibility			a. Options to customize the UI (e.g. font size, dark mode).	
			b. Polished UI with smooth transitions and animation.	

Appendix 3: Wireframes

The stick figure / character editor layout design (**Figure 7**) is based on the design from Pivot Animator, as it is a simple design. On the left will be a small toolbar for making the figure, which will have various buttons to help manipulate the stick figure or add segments. On the right will be the small canvas where the figure will appear which you can manipulate visually.

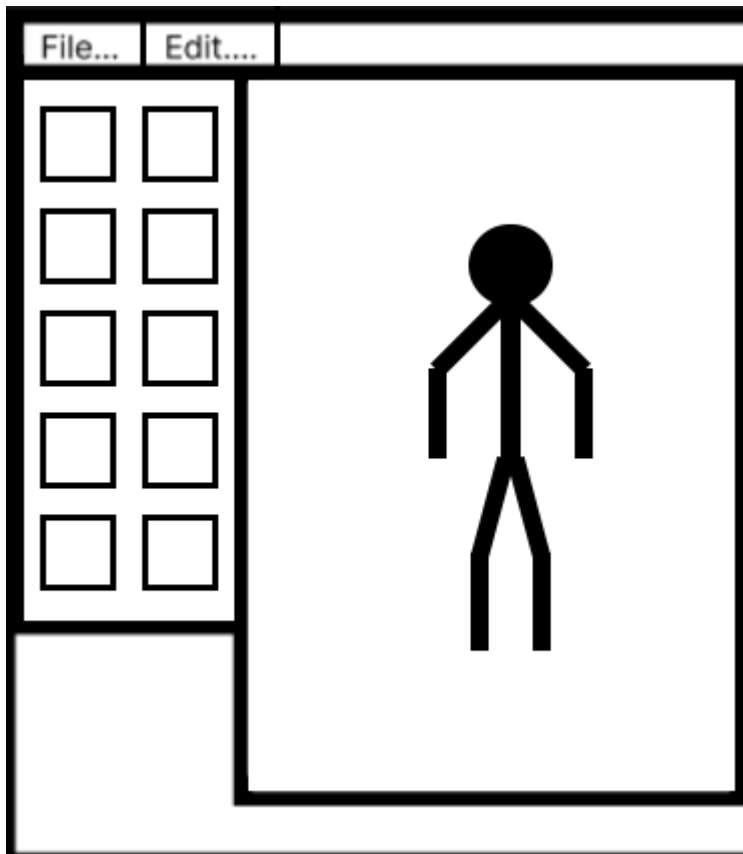


Figure 7: Wireframe of the Character Editor

Appendix 4: Requirements Assessment Table

Table 5:

Req ID	Category	Requirement Description	MoSCoW	Test Condition/Action	Expected Result	Actual Result	Status
F1.a	Interface	Provide a GUI with middle focused on graphics, sides/top for user buttons.	Must	Launch application and observe layout.	Canvas central, Toolbar left, Timeline/Menu top.	Layout matches description (Canvas center, Toolbar left, Timeline/Menu top).	Pass
F2.a	2D Animation Creation	Enable users to edit pre-built stick figures by moving joints.	Must	Select 'Hand' tool, click and drag figure points (e.g., elbow).	Figure points move, connected segments follow.	Points can be dragged to pose the figure; connected segments update visually.	Pass
F2.b	2D Animation Creation	Ability to animate stick figures using keyframes.	Must	Go to frame, pose figure, click "Add Keyframe". Go to another frame, repeat. Move playhead.	Keyframes appear on timeline, figure pose updates when scrubbing.	Keyframes can be added/removed/moved. Poses are stored and recalled correctly when changing frames.	Pass
F2.c	2D Animation	Support animating	Must	Add 2+ figures ("Add	Each figure has its own	Multiple figures can be	Pass

	Creation	multiple stick figures in the same project.		Figure" button). Animate each on its layer.	timeline row, and can be animated independently.	added, each gets a dedicated timeline layer, independent animation works.	
F2.d	2D Animation Creation	Enable real-time playback of animations within the software.	Must	Press the play button, and see if the animation plays	The animation plays in real-time	The play button plays the current animation at the desired framerate.	Pass
F2.e	2D Animation Creation	Add features like a path-based interpolation editor.	Should	Attempt to find path editing tools for interpolation.	UI element exists to define custom motion paths.	Only predefined easing (Linear, Quad In/Out/InOut) available via context menu.	Fail
F2.f	2D Animation Creation	Add onion skinning to preview the previous and next keyframes (relative to current keyframe).	Should	Press the enable onion skinning button.	The previous and next keyframes should be visible as an overlay.	The previous keyframe is visible as a red overlay, the next keyframe is visible as a green overlay.	Pass
F2.g	2D Animation Creation	3D Camera for easy view.	Could	Look for camera controls in the UI.	Ability to rotate/pan view in a 3D-like manner.	The UI is strictly 2D.	Fail
F3.a	Inverse Kinematics (IK)	Include built-in IK solver (FABRIK).	Must	Select 'Hand' tool, click and drag the end-effector	Entire limb (e.g, arm/leg) adjusts	Dragging hands/feet correctly positions the	Pass

				(hand/foot).	realistically to reach the target.	entire limb using the implemented FABRIK solver.	
F4.a	Exporting Animations	Support exporting animations in MP4 and GIF formats.	Must	Use File->Export menu for MP4 and GIF.	Playable MP4 and GIF files are created.	MP4 and GIF export functional.	Pass
F4.b	Exporting Animations	Include settings for export quality, frame rate.	Could	Check Export dialogs or Project Settings for quality options.	Options available to set resolution/quality/FPS.	Export uses default canvas size and the project FPS which is editable in Project Settings.	Partial Pass
F5.a	Physics System	Simulate simple effects like gravity to enhance realism.	Could	Look for physics enabled options or observe figure behavior.	Figures can optionally be affected by gravity.	No physics simulation (gravity or otherwise) is implemented.	Fail
F5.b	Physics System	Provide a toggle to enable/disable physics.	Could	Look for a physics toggle in UI.	A checkbox/button exists to control physics.	No physics system implemented.	Fail
F6.a	Customization	Allow users to adjust skeletons/create custom figures.	Should	Attempt to find a figure editor or skeleton modification UI.	UI exists to add/remove/resize points/segments.	Only pre-built stick figures can be added; no editor provided.	Fail

NF1. a	Performance	Software must run efficiently on average consumer hardware.	Must	Run application on target spec machine (e.g., 8GB RAM, iGPU).	UI is responsive during typical use (posing, playback).	Application runs smoothly on typical hardware for basic tasks.	Pass
NF1. b	Performance	IK calculations should be real-time and responsive.	Must	Select 'Hand' tool, drag IK end-effectors rapidly.	Figure pose updates immediately with no noticeable lag.	IK updates are visually real-time and responsive during point dragging.	Pass
NF1. c	Performance	Handle projects with multiple stick figures without significant loss.	Should	Add 5+ figures, add multiple keyframes, initiate playback.	UI remains responsive, playback maintains frame rate.	Performance is good, no noticeable lag.	Pass
NF2. a	Usability	Interface simple enough for users with no animation experience.	Must	Perform basic workflow: add figure, add keyframes, play.	Users can complete basic animation tasks without prior knowledge.	The core workflow is intuitive for basic animation tasks.	Pass
NF2. b	Usability	Provide visual cues for adjusting joints.	Should	Select tools ('Hand'/'Move'), drag points on canvas.	Clear highlighting of selected points, cursor changes contextually.	Selected points highlight gold, cursor changes (Hand/Move/Closed	Pass

						Hand).	
NF3. a	Reliability	Auto-save functionality to avoid data loss.	Must	Enable auto-save (implicitly on save), wait, force close/reopen.	Project state is restored close to the state before closing.	Auto-save implemented; saves to the current project file at intervals (requires initial save).	Pass
NF3. b	Reliability	System should not crash during common operations (save/export).	Should	Perform save, save-as, and export operations repeatedly.	Application remains stable and does not crash.	Save, save-as, and export operations are stable under normal use.	Pass
NF3. c	Reliability	Modular codebase for easier future updates/additions.	Should	Review project structure and class responsibilities.	Code follows MVC pattern, clear separation of concerns.	Application adheres to MVC architecture, promoting modularity.	Pass
NF4. a	Accessibility	Options to customize UI (font size, dark mode).	Could	Explore settings/menus for UI customization.	Options exist to change theme/fonts.	UI uses a fixed dark theme; no customization options.	Fail
NF4. b	Accessibility	Polished UI with smooth transitions and animation.	Could	Use the application, observe UI element interactions/feedback.	UI elements have smooth visual transitions or	UI is functional and clean but lacks significant	Fail

				edback.	animations.	visual polish or animated transitions.	
--	--	--	--	---------	-------------	--	--

Appendix 5: Risk Analysis

Table 6:

Risk ID	Risk Title	Risk Description	L	S	E	Mitigation	Prevention
1	Technical risk	Having trouble implementing animation capabilities or the FABRIK IK solver.	2	5	10	research other implementations, begin with simpler IK systems, seek advice from online forums, allocate extra time for debugging.	There is no real way to completely prevent the difficulty of implementing the features.
2	Time under-estimation	Underestimating the amount of time needed can result in delays in project milestones.	2	3	9	Create buffer times in the project plan and use MOSCOW prioritization to concentrate on the things that are absolutely necessary first.	Overestimate how long it could take to complete each task so there's enough time to complete it.
3	Software bugs	Unexpected issues that could impact essential functions like exporting, animation playback, or the IK system.	3	4	12	Test the essential components on a regular basis, and if additional modules are added, test them as well. Use GIT to keep version control.	It's incredibly unlikely to write perfect code without errors or bugs, so it's not feasible to prevent this completely. I can only mitigate.
4	Hardware limitations	Performance issues with the typical consumer hardware .	1	4	4	Optimize the code for efficiency.	By regularly testing on different hardware configurations, I can make sure that there won't be hardware limitations when the testing phase occurs.
5	Scope creep	Scope creep is the addition of unneeded features that hinder	2	3	6	If the feature seems unnecessary while implementing, I can use	Adhere to the specifications established with MOSCOW; concentrate on the

		development.				GIT to go back safely.	requirements that are really necessary (Must have and should have requirements take priority).
6	Personal constraints	Personal constraints include things like getting sick or other modules taking precedence.	3	4	12	Use the Gantt chart to account for these unforeseen circumstances, and let the supervisor know if delays are anticipated.	There's no way to prevent all personal constraints that could show up.

Key: L = Likelihood, S = Severity, E = Exposure

Appendix 6: Initial Project Brief (Progress Report)

Problem

Animation has long been used as a storytelling medium, and its appeal is only growing. The need for animation tools have also begun to increase. Hobbyists, students, and independent artists are unable to access affordable easy-to-use versions of these resources. Current home animation packages lack useful features including inverse kinematics or simple physics systems that could enable users to produce fluid, natural movement with more intuitive animation control. Without these features, animators who don't have access to professional software may find it difficult to create realistic, dynamic motions. The difficulty in producing animations could also limit potential creators from exploring and becoming story-tellers.

Goals

The goal of this project is to develop a 2D animation package that will feature inverse kinematics in an easy-to-use interface, making it easier for anyone with any skill level to create smooth, natural motions with characters built from simple shape tools. This could make it easier for 2D animation to become approachable for a broader audience. The software will let users export their projects in formats including MP4 and GIFs. This will allow them to easily share their work. Additionally, a stretch goal can be to investigate incorporating a basic physics system (e.g. gravity simulation) to enhance realism where desired by the animator. This could make their animations seem more grounded in reality.

Scope

This project will focus on developing a 2D animation toolkit that will let users create 2D animations, with inverse kinematics being an option for easier character control. This will be suitable for both stick-figure and simple puppet-style animations. Physics such as gravity or basic collision handling will be explored as a stretch goal, but more complex physics

elements such as momentum, or complex force simulations, will be beyond this project's scope. While this project may include basic gravity simulation, its role will be secondary and optional to the main IK system. While commercialization may be considered, the primary focus will be to create a functional prototype that could be shared for feedback. If successful, there is potential for future commercial development.